



**POLITECNICO  
DI TORINO**

Dipartimento  
di Automatica e Informatica

# Gli algoritmi di visita dei grafi

Gianpiero Cabodi e Paolo Camurati

---



## Algoritmi di visita

Visita di un grafo  $G=(V, E)$ :

- a partire da un vertice dato, seguendo gli archi con una certa strategia, elencare i vertici incontrati, eventualmente aggiungendo altre informazioni.

Algoritmi:

- in profondità (depth-first search, DFS)
- in ampiezza (breadth-first search, BFS).

## Visita in profondità (versione base)

Dato un grafo (connesso o non connesso), a partire da un vertice  $s$ , visita **tutti** i vertici del grafo (raggiungibili da  $s$  e non).

Principio base della visita in profondità: espandere l'ultimo vertice **scoperto** che ha ancora vertici non ancora scoperti adiacenti.

 **visitarlo la prima volta**

Scoperta di un vertice: prima volta che si incontra nella visita (discesa ricorsiva, visita in pre-order). Il vettore `pre` gioca il ruolo del vettore `mark` nel Calcolo Combinatorio, ma contiene i tempi crescenti di scoperta.

opzione 1 (ampiezza) :tutti i vertici raggiungibili

opzione 2 (profondità) : TUTTI i vertici, anche quelli non raggiungibili

## Algoritmo

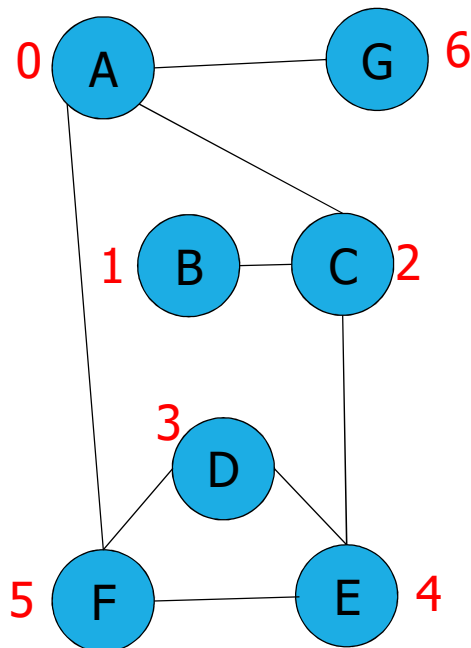
wrapper

- `GRAPHsimpleDfs`: funzione che, a partire da un vertice dato, visita tutti i vertici di un grafo, richiamando la procedura ricorsiva `SimpleDfsR`. Termina quando tutti i vertici sono stati visitati.
- `SimpleDfsR`: funzione che visita in profondità a partire da un vertice `id` identificato con un arco fittizio `EDGEcreate(id, id)` utile in fase di visualizzazione. Termina quando ha visitato in profondità tutti i nodi raggiungibili da `id`.

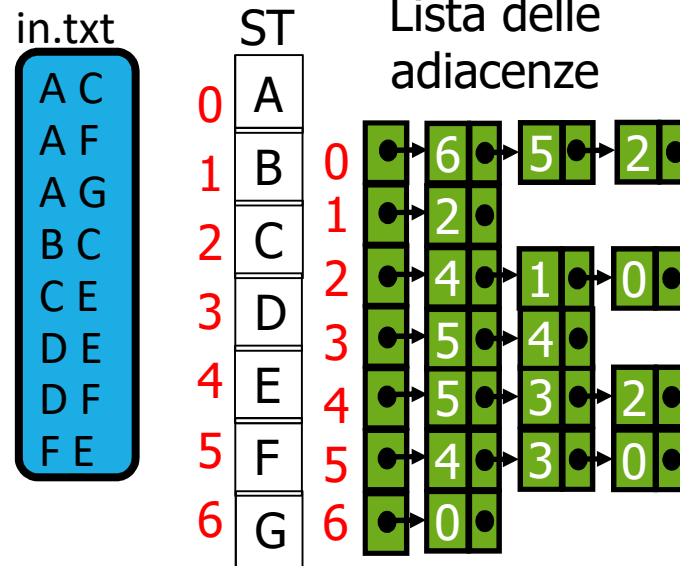
B: alcuni autori chiamano visita in profondità la sola `SimpleDfsR`.

ordine di visita dipende dalla lista delle adiacenze

## Esempio

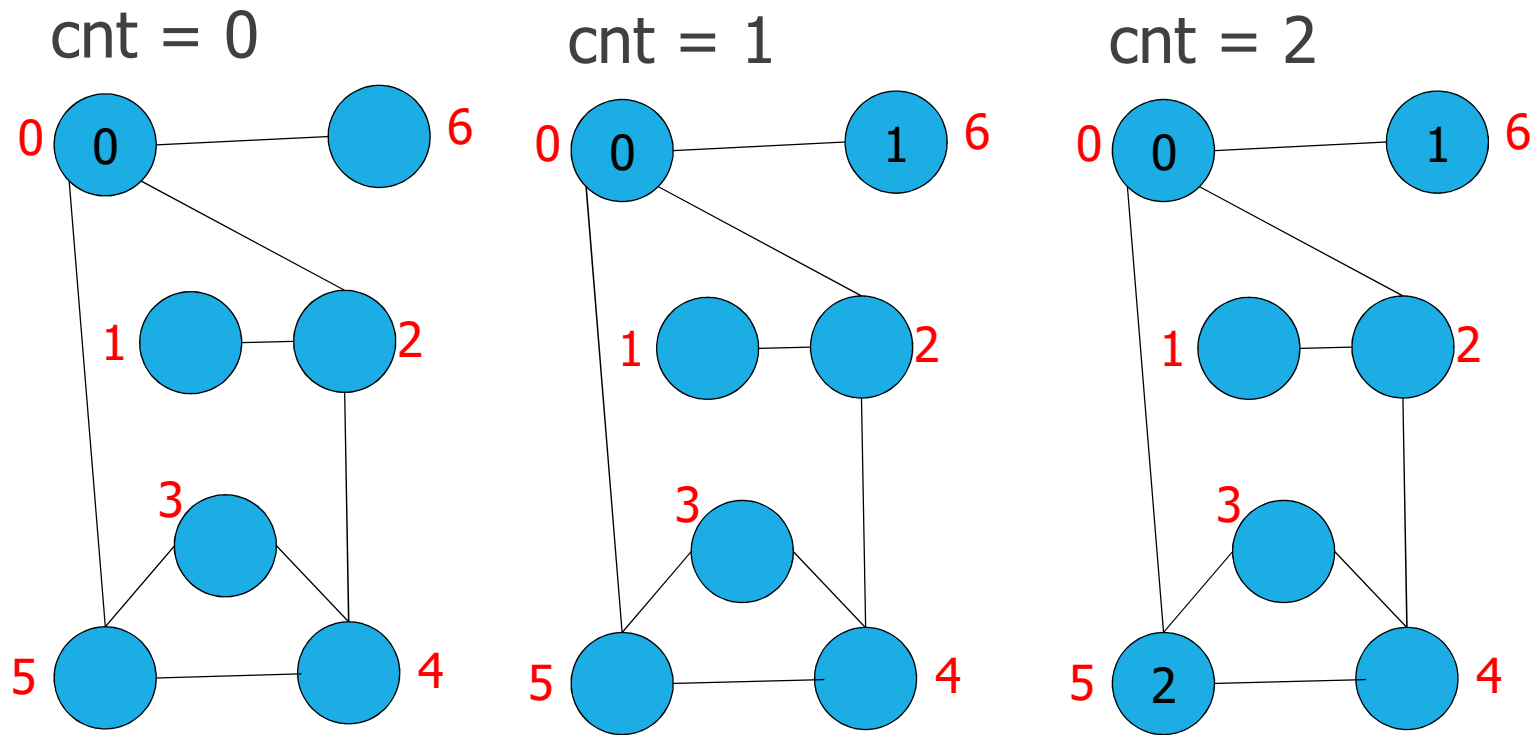


parto dal nodo 0: esso è connesso al nodo 6 "per primo" dato che all'indice 0 il primo elemento che appare (+ a sinistra) è il 6



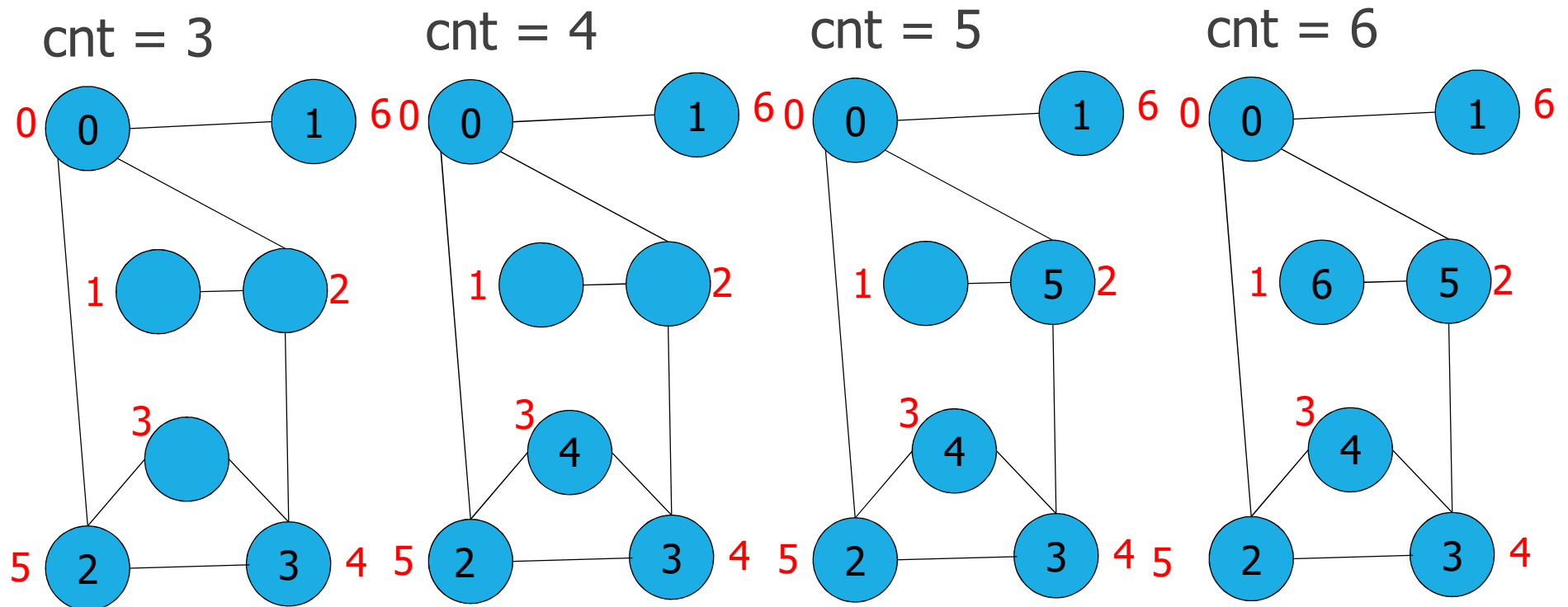
## Esempio

non posso scegliere oltre, torno al nodo 0 e  
scendo da un'altra parte



scendo ricorsivamente sul  
vertice 5 e cerco quelli a  
cui collega

Il vertice 2 lo scopro dal vertice 4



## Strutture dati

- grafo non pesato come lista delle adiacenze
- vettore `pre[i]` dove per ciascun vertice si registra il tempo di scoperta (numerazione in preordine dei vertici)
- contatore `cnt` per tempi di scoperta

`cnt` e `*pre` sono locali alla funzione `GRAPHsimpleDfs` e passati by reference alla funzione ricorsiva `SimpleDfsR`.



```

void GRAPHsimpleDfs(Graph G, int id) {
    int v, cnt=0, *pre;
    pre = malloc(G->V * sizeof(int));
    if ((pre == NULL)) return;
    for (v=0; v<G->V; v++) pre[v]=-1;
    simpleDfsR(G, EDGEcreate(id,id), &cnt, pre);

    for (v=0; v < G->V; v++)
        if (pre[v]== -1)
            simpleDfsR(G, EDGEcreate(v,v), &cnt, pre);

    printf("discovery time labels \n");
    for (v=0; v < G->V; v++)
        printf("vertex %s : %d \n", STsearchByIndex(G->tab, v), pre[v]);
}

```

visita a partire da id

visita dei nodi non ancora visitati

```
static void simpleDfsR(Graph G, Edge e, int *cnt, int *pre) {  
    link t; int w = e.w;  
    pre[w] = (*cnt)++;  
    for (t = G->ladj[w]; t != G->z; t = t->next)  
        if (pre[t->v] == -1)  
            simpleDfsR(G, EDGEcreate(w, t->v), cnt, pre);  
}
```

terminazione implicita  
della ricorsione

esaurendo la lista di vertici adiacenti termino la ricorsione

## Visita in profondità (versione estesa)

Estensione:

- nodi etichettati con etichetta tempo di scoperta / tempo di completamento  tempo del suo completamento

 foresta di alberi della visita in profondità, memorizzata in un vettore.  
i grafi non sono necessariamente connessi

Scoperta di un vertice: prima volta che si incontra nella visita (discesa ricorsiva, visita in pre-order), vettore  $pre[i]$ .

Completamento: fine dell'elaborazione del vertice (uscita dalla ricorsione, visita in post-order) vettore  $post[i]$ .

Scoperta/Completamento: tempo discreto che avanza mediante contatore  $time$ . Avanzamento quando si scopre o si completa.

Identificazione del padre nella visita in profondità: vettore  $st[i]$ .

## Algoritmo

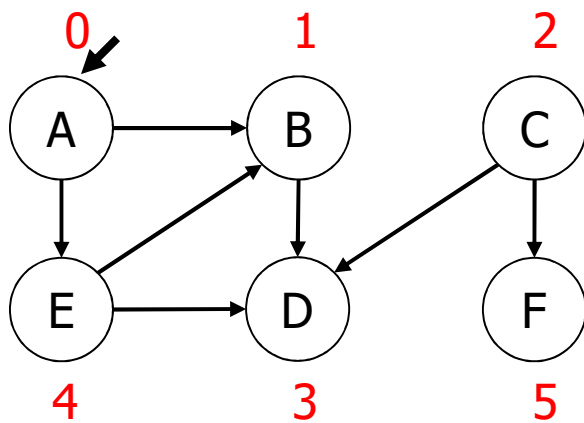
wrapper

- `GRAPHextendedDfs`: funzione che, a partire da un vertice dato, visita tutti i vertici di un grafo, richiamando la procedura ricorsiva `ExtendedDfsR`. Termina quando tutti i vertici sono stati visitati.
- `ExtendedDfsR`: funzione che visita in profondità a partire da un vertice `id` identificato con un arco fittizio `EDGEcreate(id, id)` utile in fase di visualizzazione. Termina quando ha visitato in profondità tutti i nodi raggiungibili da `id`.

B: alcuni autori chiamano visita in profondità la sola `ExtendedDfsR`.

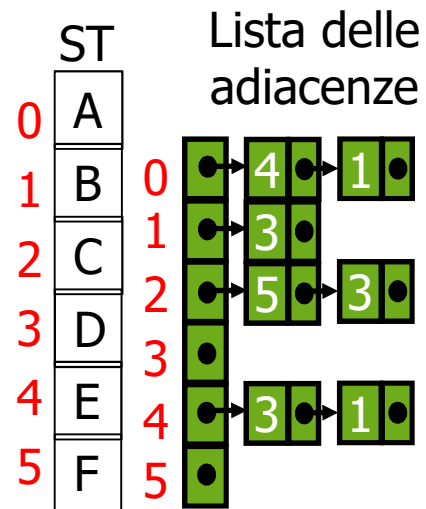
# Esempio

time = 0      st



in.txt

```
A B
C D
A E
C F
B D
E B
E D
```



|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| st | -1 | -1 | -1 | -1 | -1 | -1 |
|    | 0  | 1  | 2  | 3  | 4  | 5  |

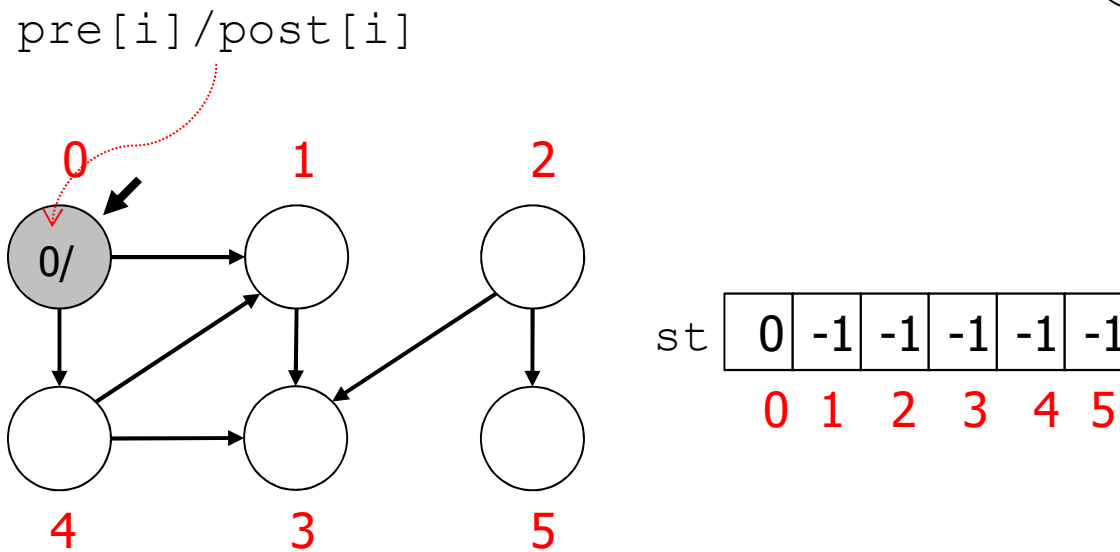
un nodo è terminato se non ci sono più nodi  
adiacenti ad esso NON ancora scoperti

Convenzione grafica:

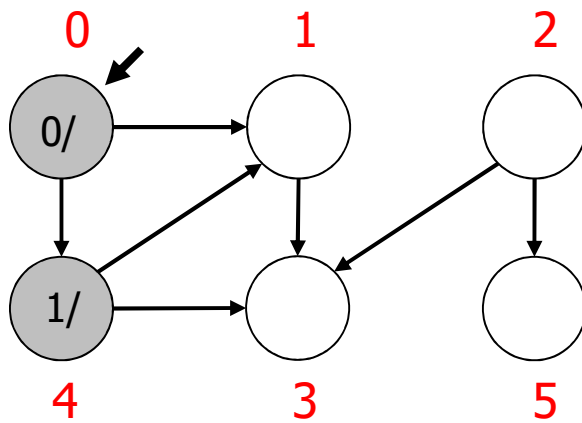
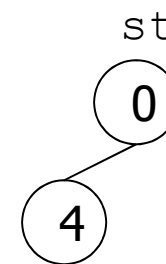
- nodo bianco: non ancora scoperto
- nodo grigio: scoperto ma non terminato
- nodo nero: terminato

time = 0

st  
0



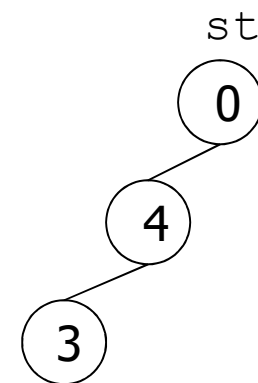
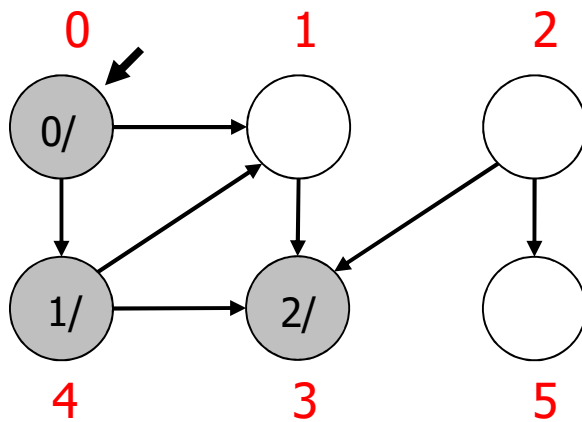
time = 1



st

|   |    |    |    |   |    |
|---|----|----|----|---|----|
| 0 | -1 | -1 | -1 | 0 | -1 |
| 0 | 1  | 2  | 3  | 4 | 5  |

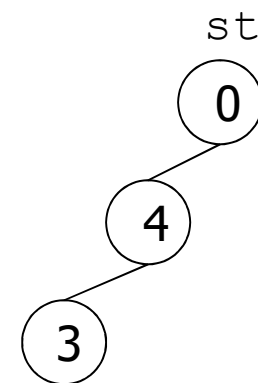
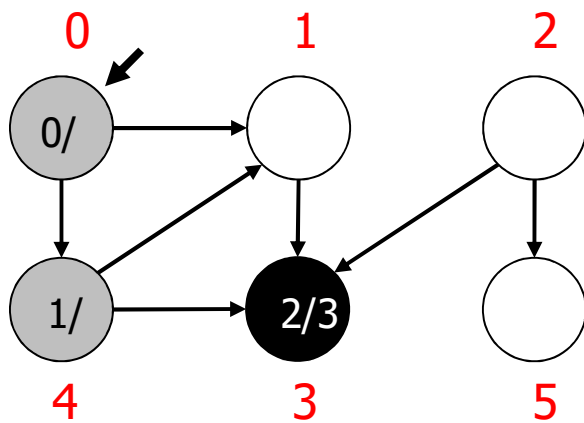
time = 2



|    |   |    |    |   |   |    |
|----|---|----|----|---|---|----|
| st | 0 | -1 | -1 | 4 | 0 | -1 |
|    | 0 | 1  | 2  | 3 | 4 | 5  |

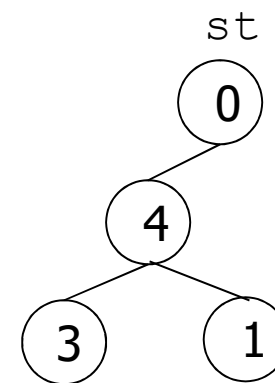
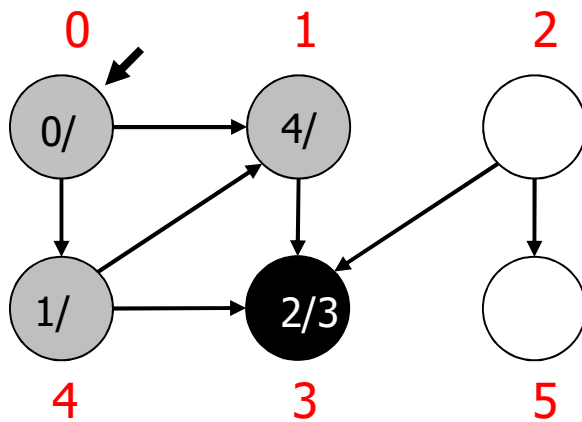


time = 3



|    |   |    |    |   |   |    |
|----|---|----|----|---|---|----|
| st | 0 | -1 | -1 | 4 | 0 | -1 |
|    | 0 | 1  | 2  | 3 | 4 | 5  |

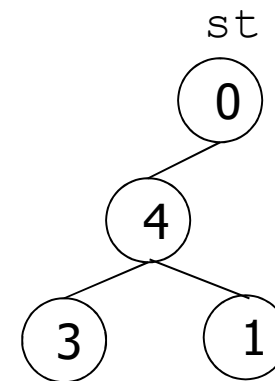
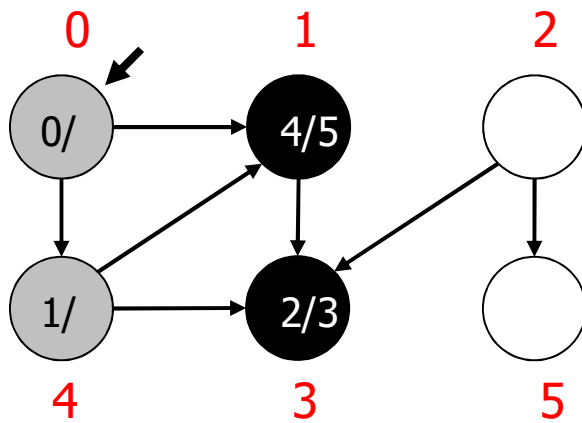
time = 4



st

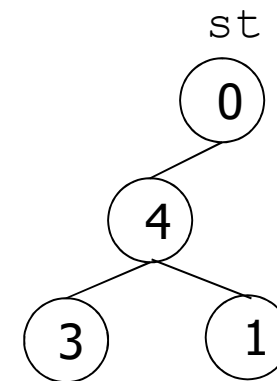
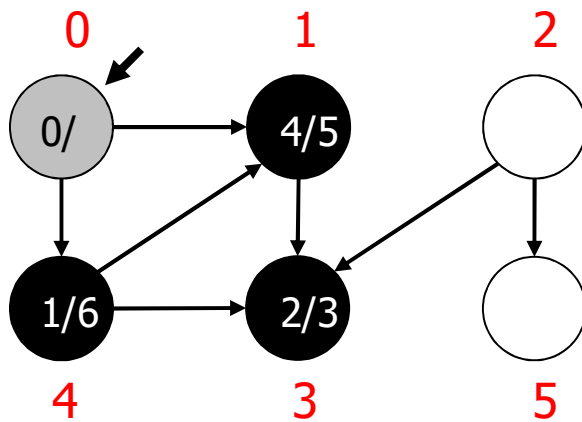
|   |   |    |   |   |    |
|---|---|----|---|---|----|
| 0 | 4 | -1 | 4 | 0 | -1 |
| 0 | 1 | 2  | 3 | 4 | 5  |

time = 5



|    |   |   |    |   |   |    |
|----|---|---|----|---|---|----|
| st | 0 | 4 | -1 | 4 | 0 | -1 |
|    | 0 | 1 | 2  | 3 | 4 | 5  |

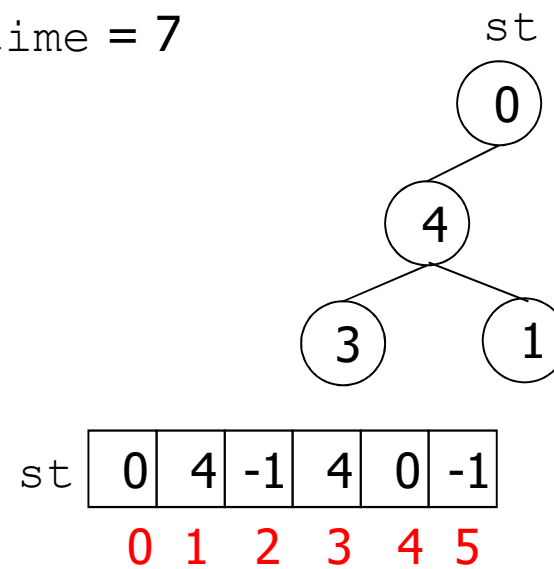
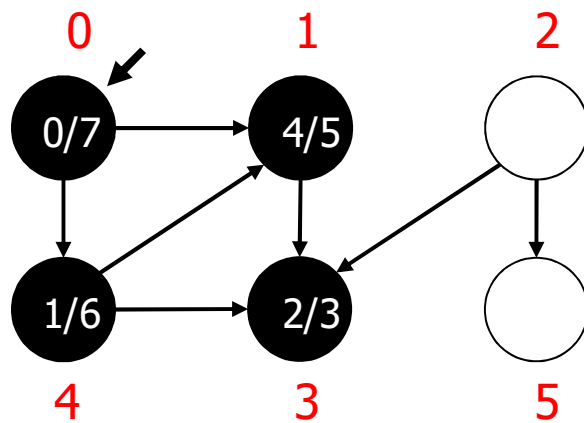
time = 6



|    |   |   |    |   |   |    |
|----|---|---|----|---|---|----|
| st | 0 | 4 | -1 | 4 | 0 | -1 |
|    | 0 | 1 | 2  | 3 | 4 | 5  |

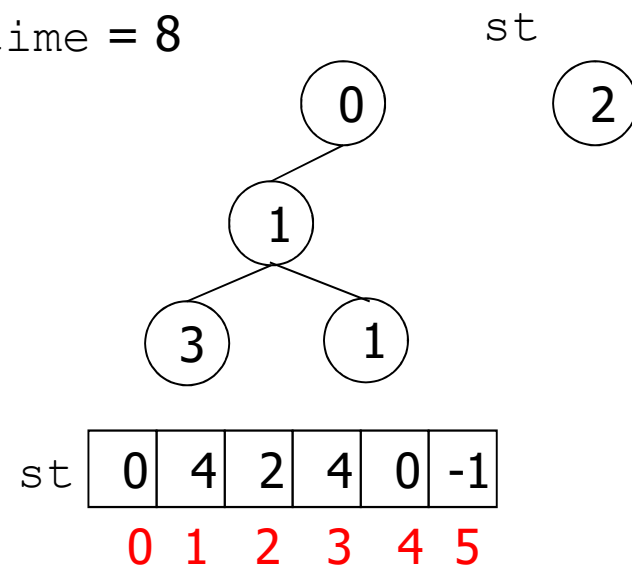
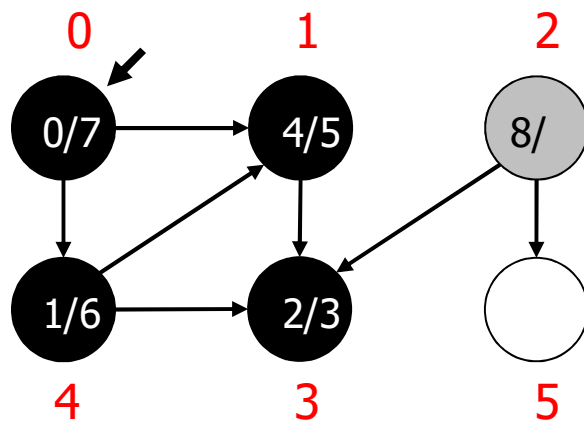
non ci sono più nodi  
raggiungibili dal  
vertice zero

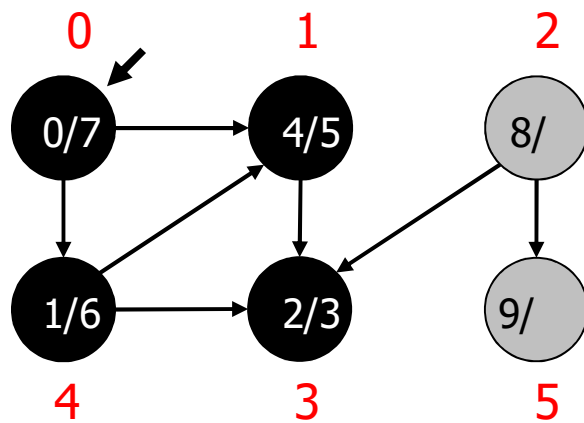
time = 7



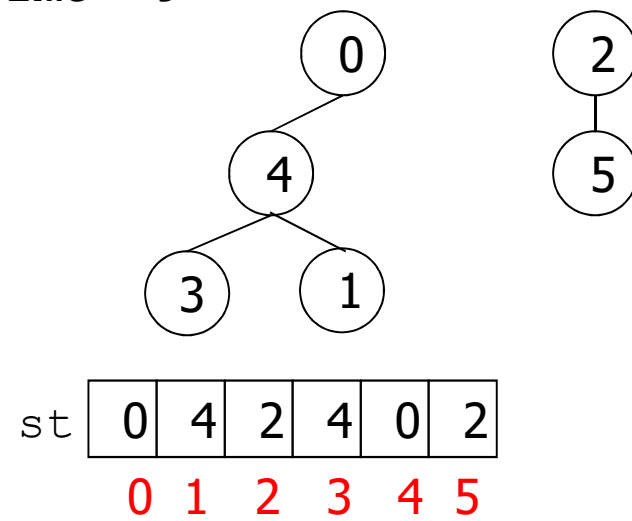
riparte dal primo nodo  
che incontra (in questo  
caso è il nodo 2)

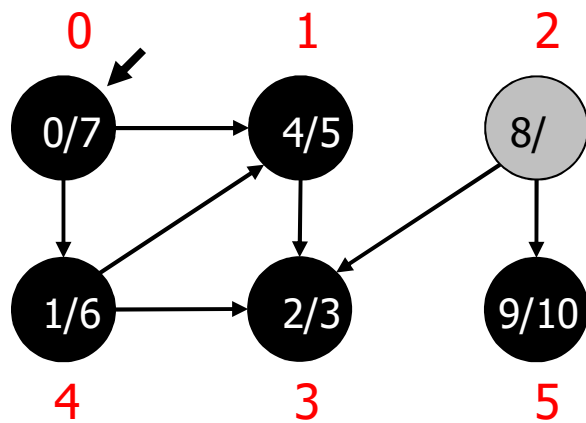
time = 8



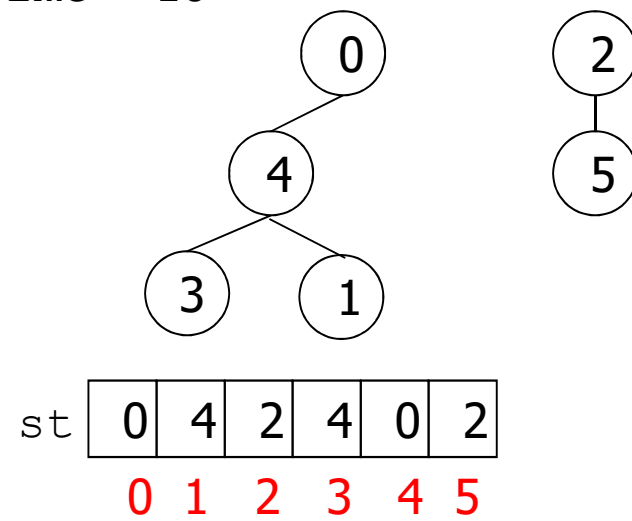


time = 9

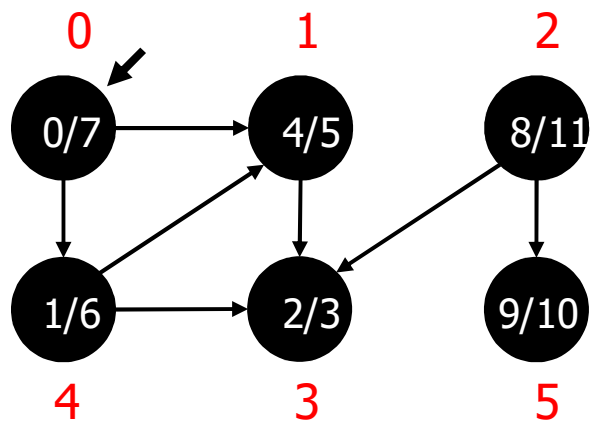




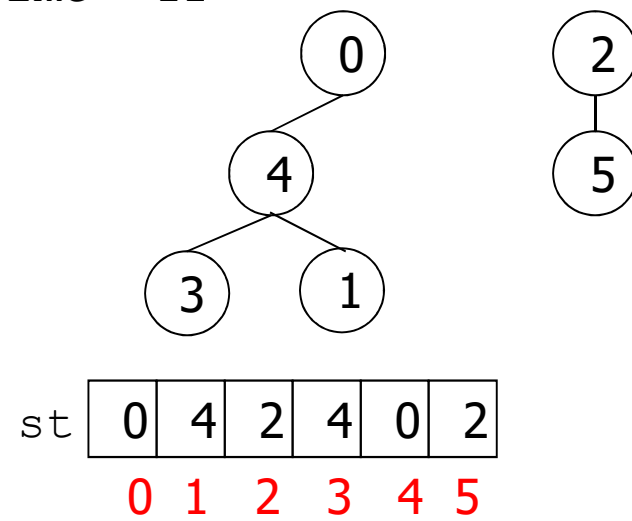
time = 10







time = 11



## Strutture dati

- grafo non pesato come lista delle adiacenze
  - vettori dove per ciascun vertice:
    - si registra il tempo di scoperta (numerazione in preordine dei vertici) `pre[i]`
    - si registra il tempo di completamento (numerazione in postordine dei vertici) `post[i]`
    - si registra il padre per la costruzione della foresta degli alberi della visita in profondità: `st[i]`
  - contatore `time` per tempi di scoperta/completamento
- `time`, `*pre`, `*post` e `*st` sono locali alla funzione `GRAPHextendedDfs` e passati by reference alla funzione ricorsiva `ExtendedDfsR`.

```

void GRAPHextendedDfs(Graph G, int id) {
    int v, time=0, *pre, *post, *st;
    pre/post/st = malloc(G->V * sizeof(int));
    for (v=0;v<G->V;v++) {
        pre[v]=-1; post[v]=-1; st[v]=-1;}
    extendedDfsR(G, EDGEcreate(id,id), &time, pre, post, st);
    for (v=0; v < G->V; v++)
        if (pre[v]==-1)
            extendedDfsR(G,EDGEcreate(v,v),&time,pre,post,st);
    printf("discovery/endprocessing time labels \n");
    for (v=0; v < G->V; v++)
        printf("%s:%d/%d\n",STsearchByIndex(G->tab,v),pre[v],post[v]);
    printf("resulting DFS tree \n");
    for (v=0; v < G->V; v++)
        printf("%s's parent: %s \n",STsearchByIndex (G->tab, v),
            STsearchByIndex (G->tab, st[v]));
}

```

```

static void ExtendedDfsR(Graph G, Edge e, int *time, int *pre,
                        int *post, int *st) {
    link t;
    int w = e.w; entro tramite arco e, ottengo vertice di indice w
    st[e.w] = e.v; il padre del nodo w è il nodo v
    pre[w] = (*time)++; ho scoperto il nodo w al tempo time
    for (t = G->ladj[w]; t != G->z; t = t->next)
        if (pre[t->v] == -1)
            ExtendedDfsR(G, EDGEcreate(w, t->v), time, pre, post, st);
    post[w] = (*time)++;
}

```

che verrà opportunamente incrementato

terminazione implicita della ricorsione

## Visita in profondità (versione completa)

Si etichetta ogni arco:

- grafi orientati: T(tree), B(backward), F(forward), C(cross)
- grafi non orientati: T(tree), B(backward)

## Classificazione degli archi

Grafo orientato:

- **T**: archi dell'albero della visita in profondità
- **B**: connettono un vertice  $w$  ad un suo antenato  $v$  nell'albero: è un arco che non permette ulteriore discesa ricorsiva, è connesso a nodi già scoperti  
tempo di fine elaborazione di  $v$  **sarà**  $>$  tempo di fine elaborazione di  $w$ .

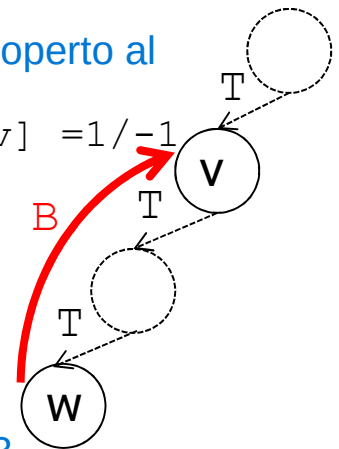
Equivale a testare se, quando scopro l'arco  $(w, v)$ ,  
 $\text{post}[v] == -1$

nodo  $v$  scoperto al tempo 1  
 $\text{pre}[v] / \text{post}[v] = 1 / -1$

è un nodo che torna indietro, al suo antenato nella visita in profondità

se il nodo a cui punta l'arco Back (B) (in qst il nodo  $v$ ) NON è ancora terminato ma il nodo da cui parte B (in questo caso nodo  $w$ ) è già terminato allora l'arco è un arco di tipo B

$\text{pre}[w] / \text{post}[w] = 3 / 4$   
nodo  $w$  scoperto al tempo 3



entrambi nodi connessi ad F sono già stati scoperti

- **F**: connettono un vertice  $w$  ad un suo discendente  $v$  nell'albero:

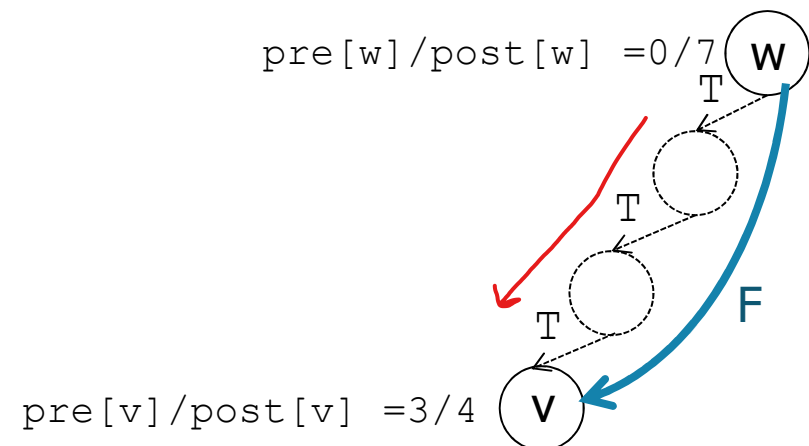
tempo di scoperta di  $w$  **è**  $>$  tempo di scoperta di  $v$  quando  
scopro l'arco  $(w, v)$

Prima scendo da  $w$  tramite nodi senza nome fino al nodo  $v$

$\text{pre}[v] > \text{pre}[w]$

$w$  è terminato DOPO  $v$

arco di tipo Forward



- **C**: archi rimanenti, per cui  
tempo di scoperta di  $w$  **è**  $>$  tempo di scoperta di  $v$  quando scopro  
l'arco  $(w, v)$

$$pre[w] > pre[v]$$

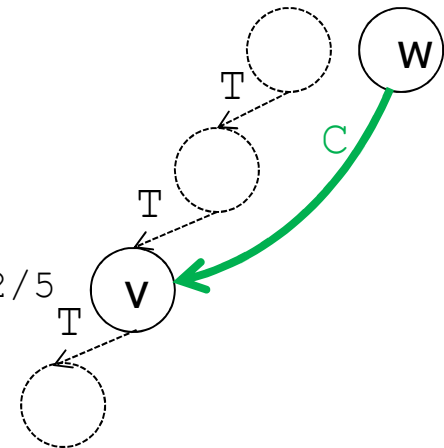
arco **Cross** che non sono nè di tipo **B** nè di tipo **F**

NON di tipo **B** perchè  $v$  è stato già scoperto e **TERMINATO**

NON di tipo **F** perchè  $pre[v] < pre[w]$

$$pre[v]/post[v] = 2/5$$

$$pre[w]/post[w] = 8/-1$$



Grafo non orientato: solo archi **T** e **B**.



## Algoritmo

wrapper

- `GRAPHdfs`: funzione che, a partire da un vertice dato, visita tutti i vertici di un grafo, richiamando la procedura ricorsiva `dfsR`. Termina quando tutti i vertici sono stati visitati.
- `dfsR`: funzione che visita in profondità a partire da un vertice `id` identificato con un arco fittizio `EDGEcreate(id, id)` utile in fase di visualizzazione. Termina quando ha visitato in profondità tutti i nodi raggiungibili da `id`.

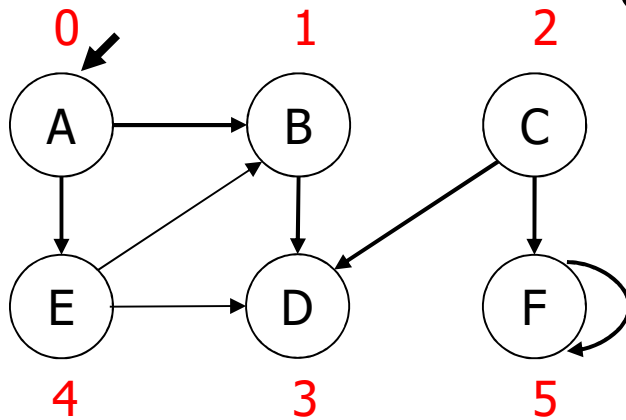
NB: alcuni autori chiamano visita in profondità la sola `dfsR`.

Strutture dati: come nella versione estesa. Codice della `GRAPHdfs` identico a quello della `GRAPHextendedDfs`.

# Esempio

grafo orientato non pesato

time = 0    st



Lista archi

|   |   |
|---|---|
| A | B |
| C | D |
| A | E |
| C | F |
| B | D |
| E | D |
| E | B |
| F | F |

| ST | 0 | 1 | 2 | 3 | 4 | 5 |
|----|---|---|---|---|---|---|
| A  |   |   |   |   |   |   |
| B  |   |   |   |   |   |   |
| C  |   |   |   |   |   |   |
| D  |   |   |   |   |   |   |
| E  |   |   |   |   |   |   |
| F  |   |   |   |   |   |   |

Lista delle  
adiacenze

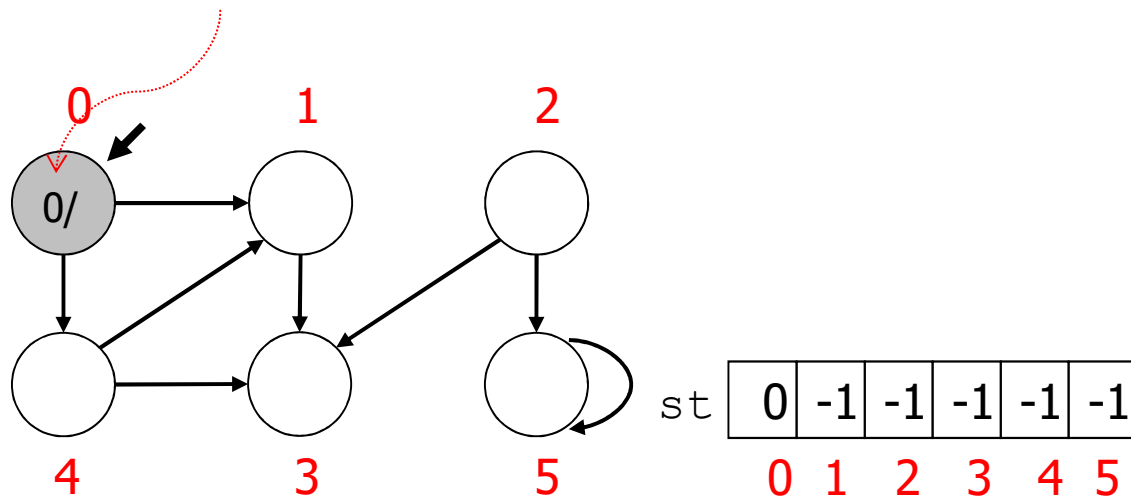
|   |   |   |
|---|---|---|
| 0 | 4 | 1 |
| 1 | 3 |   |
| 2 | 5 | 3 |
| 3 |   |   |
| 4 | 1 | 3 |
| 5 | 5 |   |

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| st | -1 | -1 | -1 | -1 | -1 | -1 |
|    | 0  | 1  | 2  | 3  | 4  | 5  |

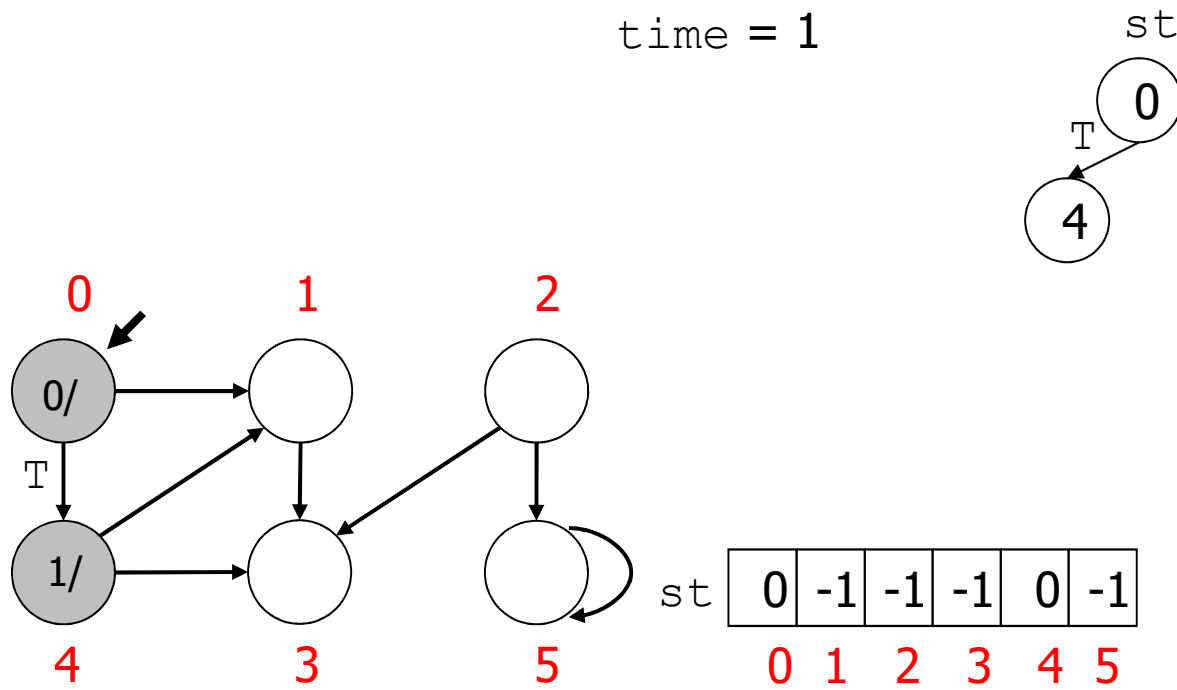
time = 0

st  
0

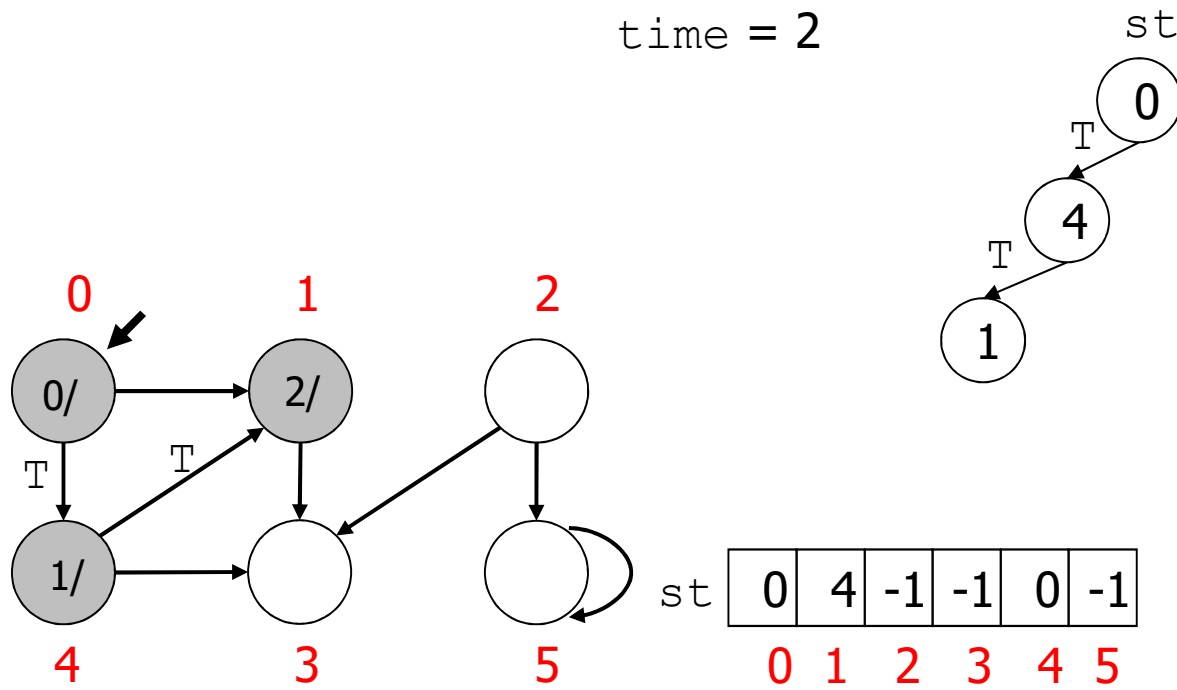
pre[i]/post[i]



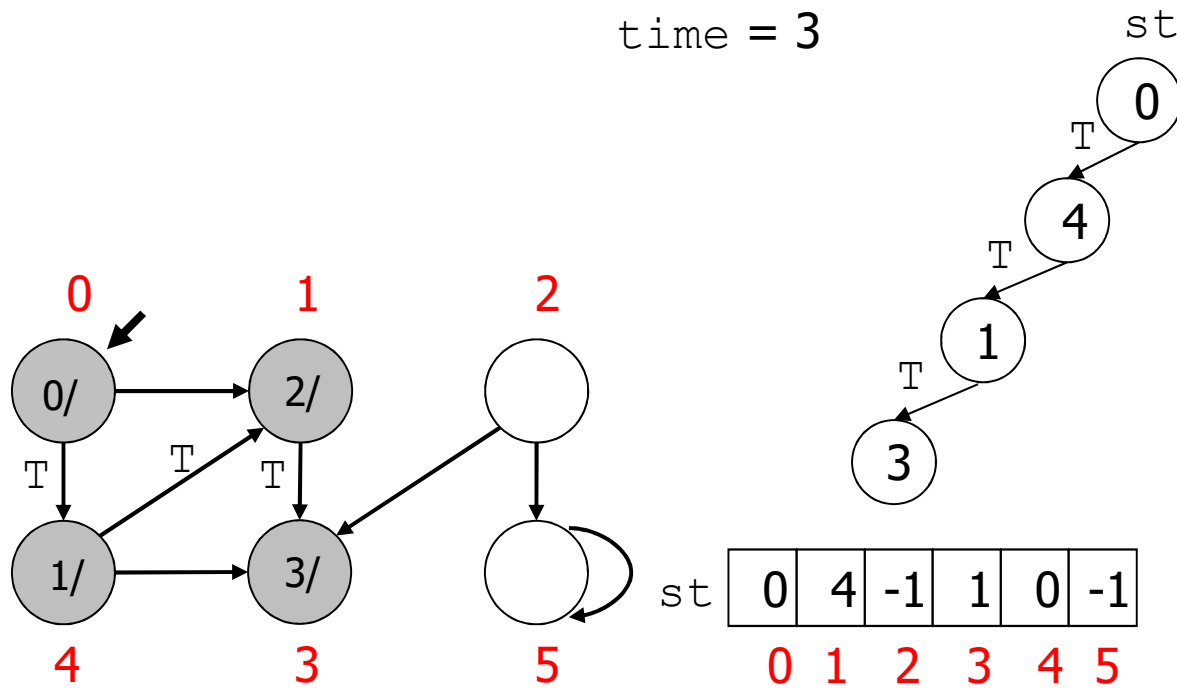
time = 1



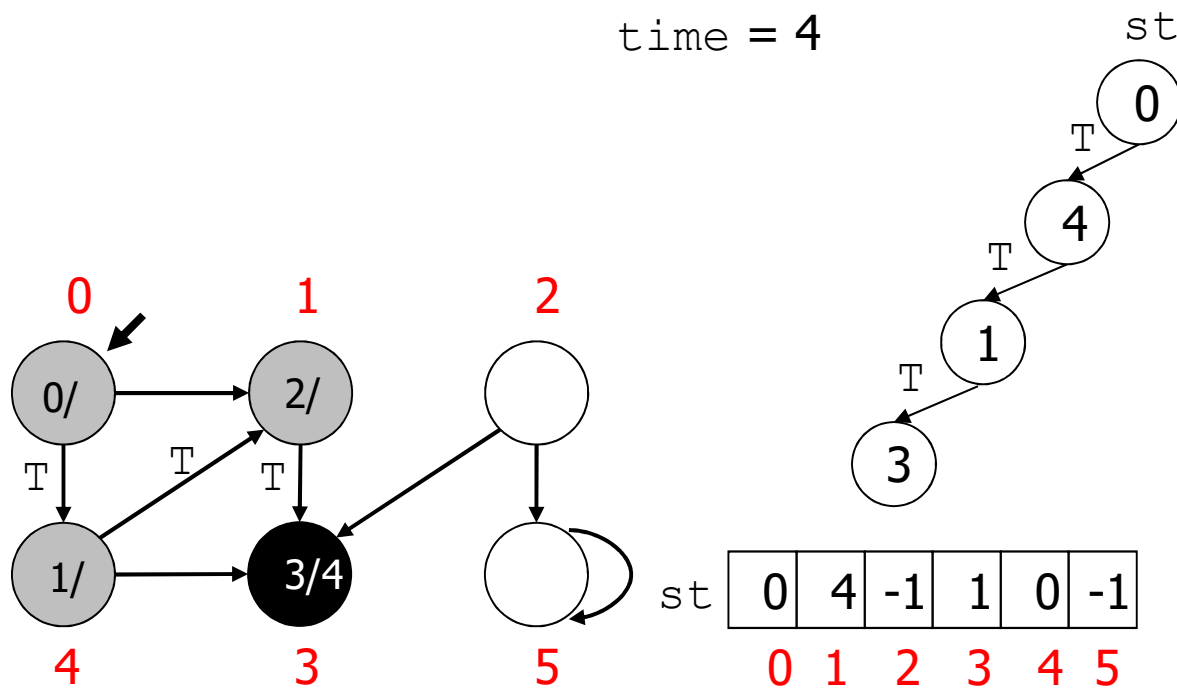
time = 2



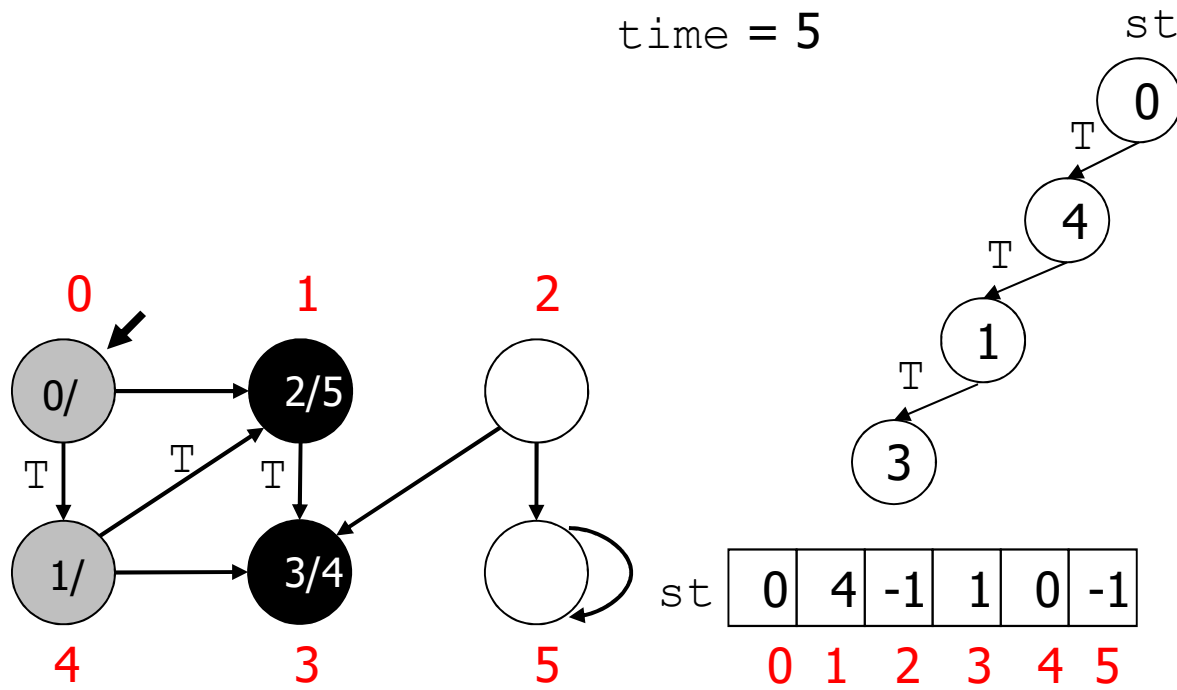
time = 3



time = 4

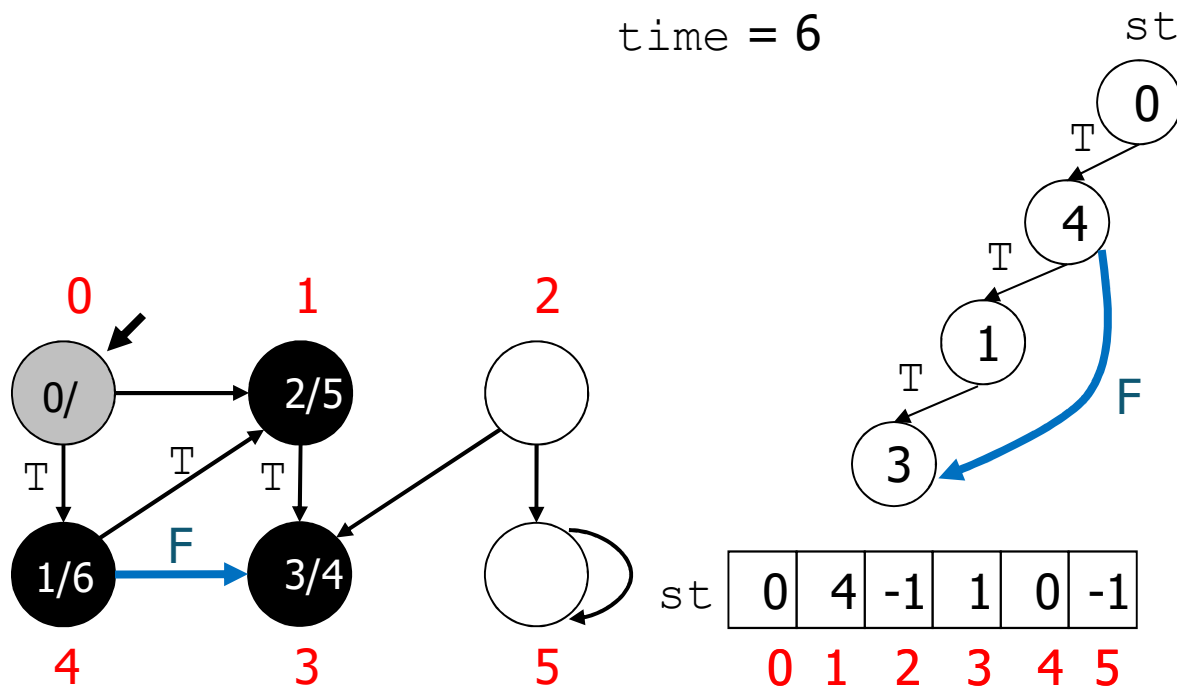


time = 5

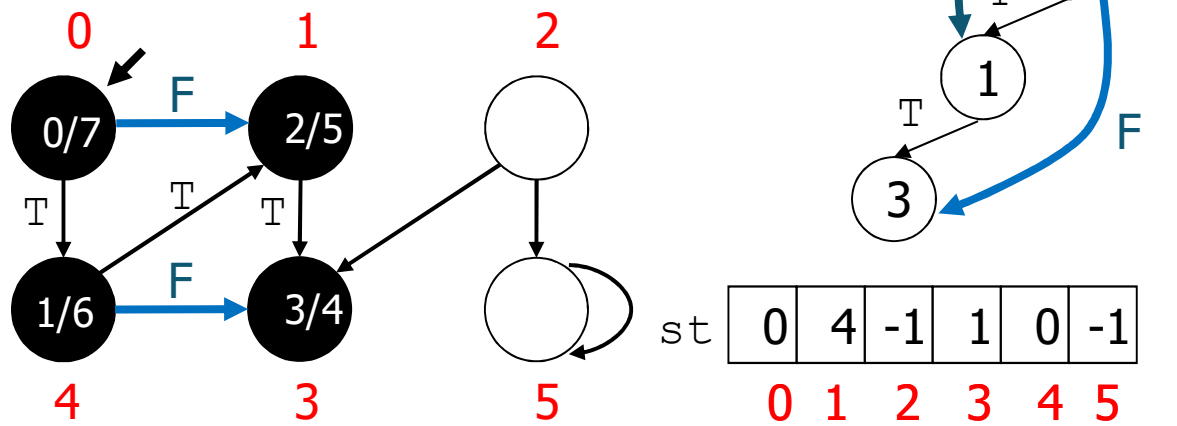




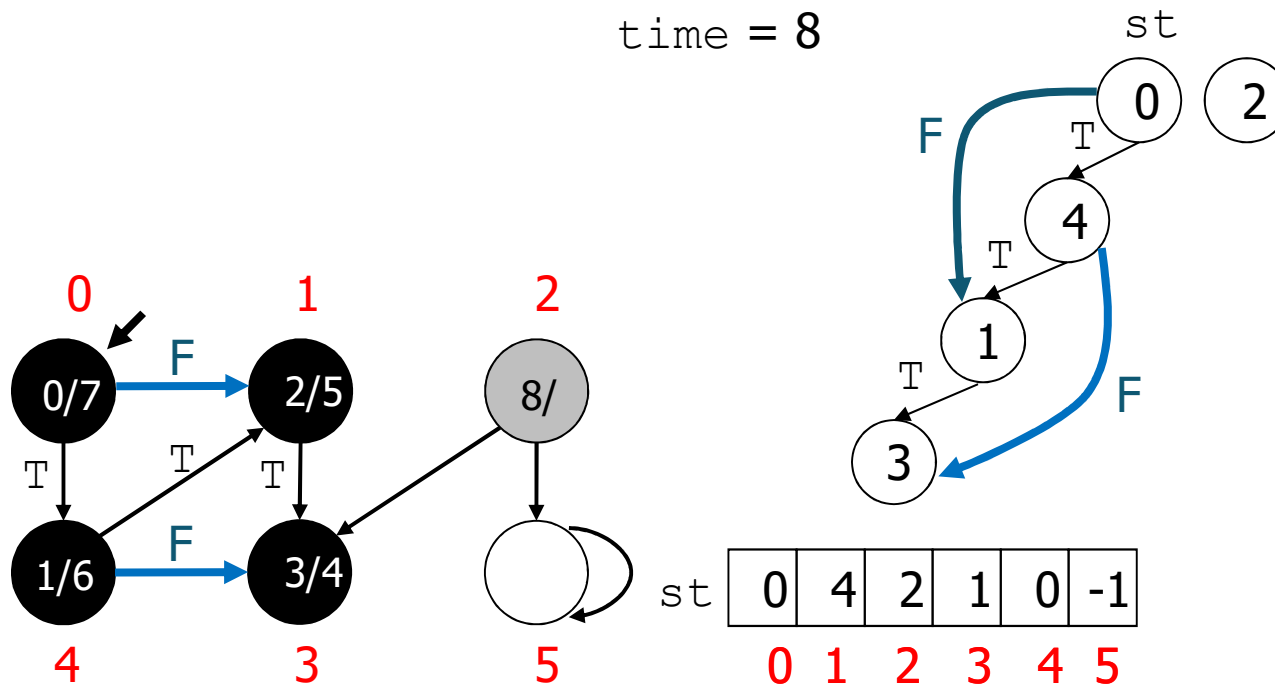
time = 6



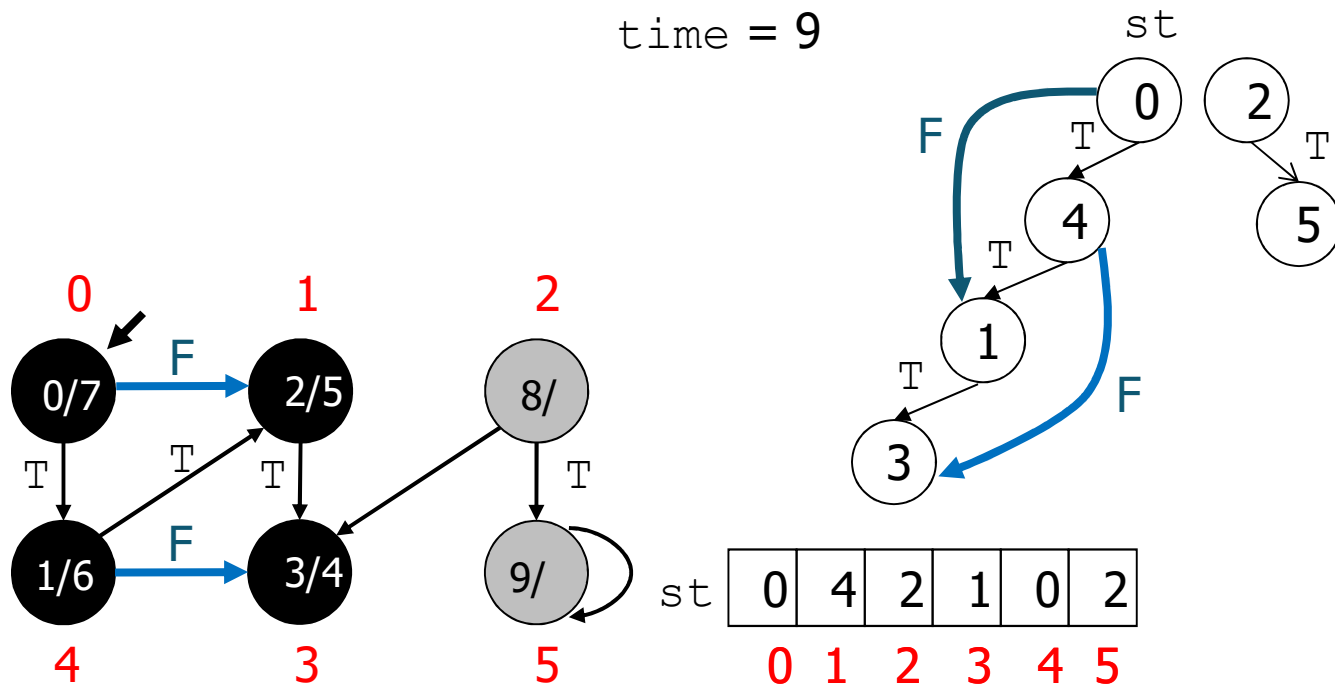
time = 7



time = 8

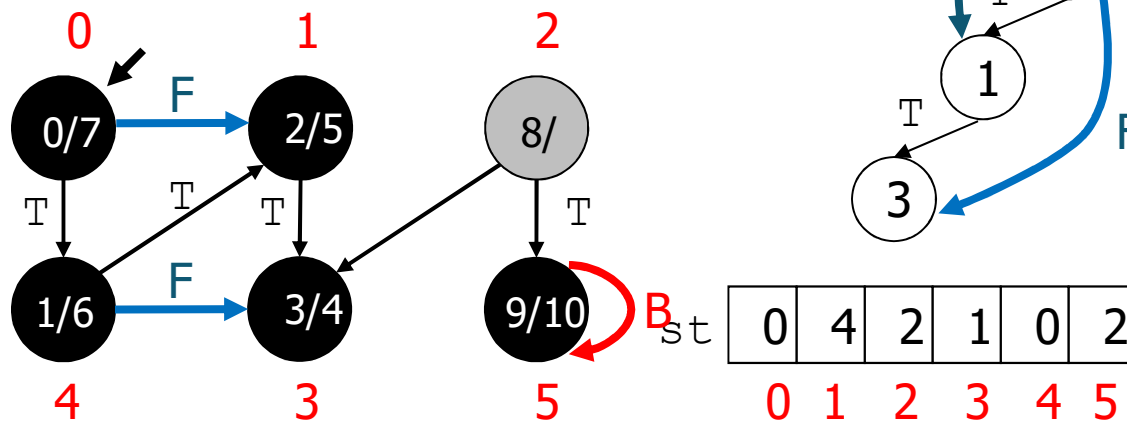


time = 9



time = 10

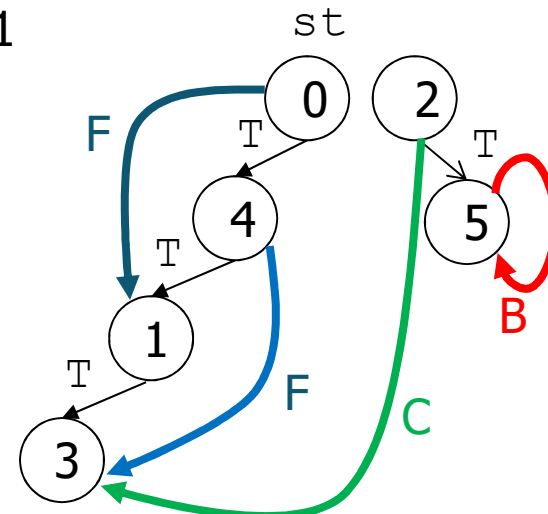
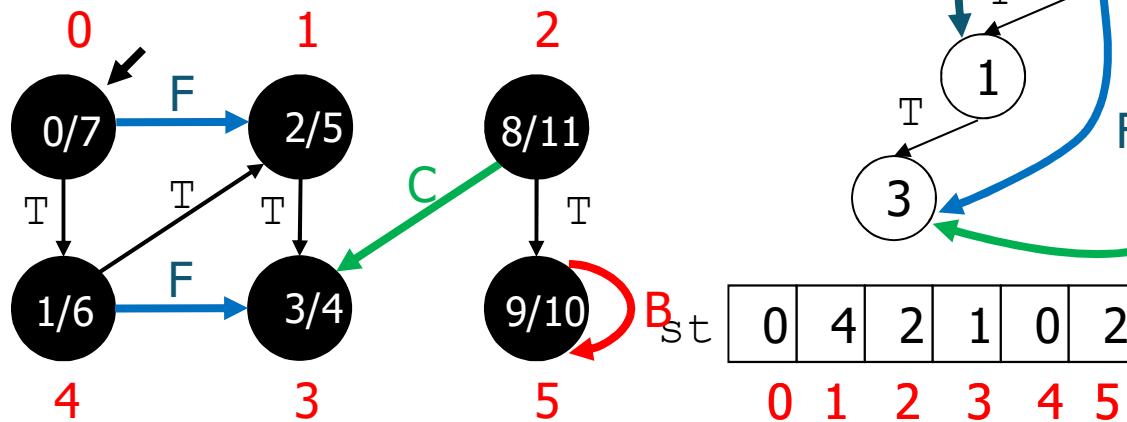
tutti i nodi su sè stessi sono di tipo back?



il selfloop (5->5) porta a un nodo che NON è  
ancora stato terminato (nodo 5)  
rendendo l'arco di tipo backward

time = 11

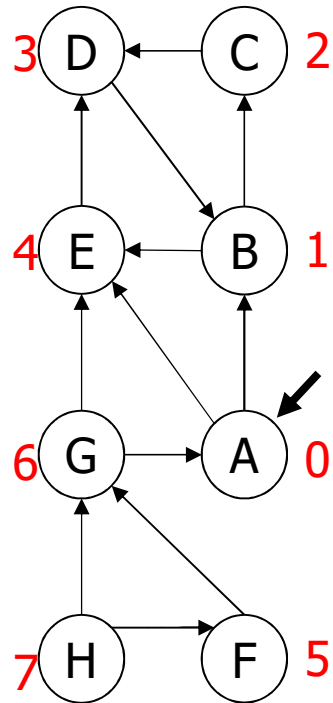
dopo aver determinato il nodo B il nodo 5 termina



# Esempio esercizio da fare

Lista archi

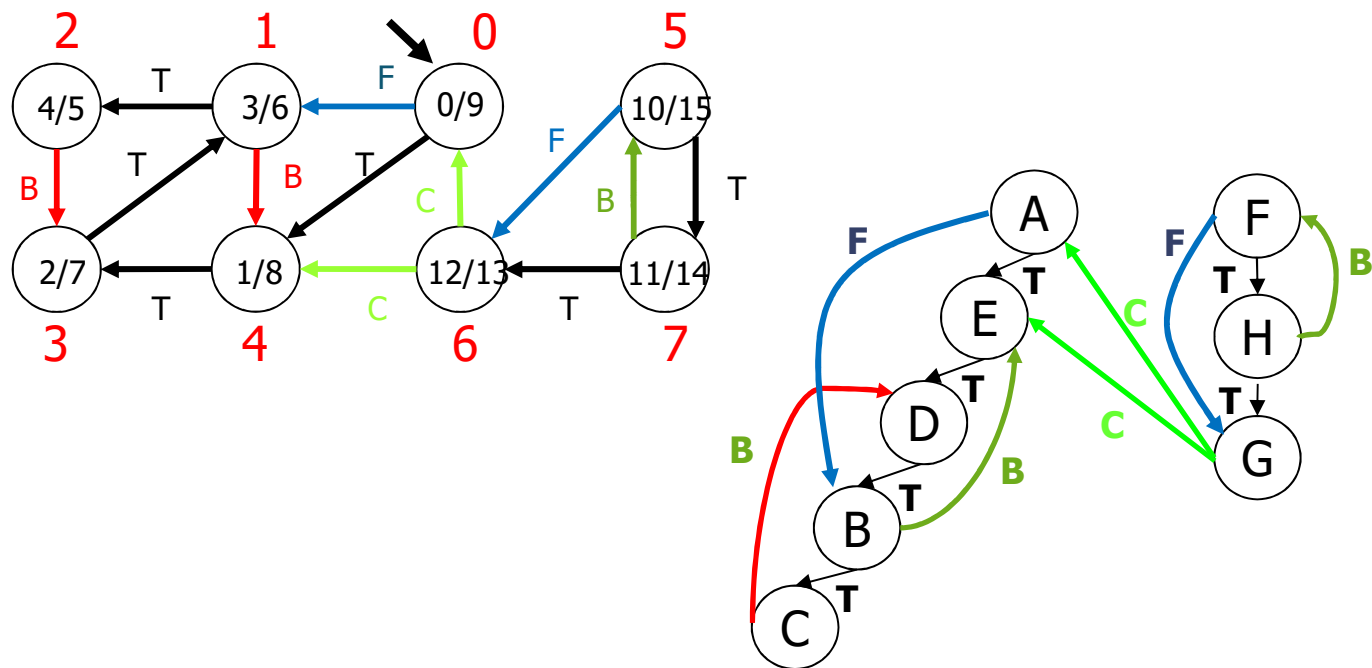
|   |   |
|---|---|
| A | B |
| B | C |
| C | D |
| A | E |
| F | G |
| F | H |
| D | B |
| E | D |
| B | E |
| G | E |
| H | G |
| H | F |
| G | A |



|   | ST |
|---|----|
| 0 | A  |
| 1 | B  |
| 2 | C  |
| 3 | D  |
| 4 | E  |
| 5 | F  |
| 6 | G  |
| 7 | H  |

Lista delle  
adiacenze

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | ● | → | 4 | ● | → | 1 | ● |
| 1 | ● | → | 4 | ● | → | 2 | ● |
| 2 | ● | → | 3 | ● | → |   |   |
| 3 | ● | → | 1 | ● | → |   |   |
| 4 | ● | → | 3 | ● | → |   |   |
| 5 | ● | → | 7 | ● | → | 6 | ● |
| 6 | ● | → | 0 | ● | → | 4 | ● |
| 7 | ● | → | 5 | ● | → | 6 | ● |





```

void dfsR(Graph G, Edge e, int *time,
          int *pre, int *post, int *st){
    link t;
    int v, w = e.w;
    Edge x;
    if (e.v != e.w) {
        printf("(s, s): T \n", STsearchByIndex(G->tab, e.v),
            STsearchByIndex(G->tab, e.w));
    }
    st[e.w] = e.v;
    pre[w] = (*time)++;
    for (t = G->ladj[w]; t != G->z; t = t->next)
        if (pre[t->v] == -1)
            dfsR(G, EDGEcreate(w, t->v), time, pre, post, st);
    else {
        v = t->v;
        x = EDGEcreate(w, v);
    }
}

```

escludi arco fittizio

terminazione implicita  
della ricorsione

se NON è di tipo T allora è di tipo B per grafi NON orientati

test per non considerare gli archi 2 volte

grafi non orientati

```
if (pre[w] < pre[v])
    printf("(%s, %s): B\n", STsearchByIndex(G->tab, x.v),
        STsearchByIndex(G->tab, x.w)) ; arco di tipo T
if (post[v] == -1)
    printf("(%s, %s): B\n", STsearchByIndex(G->tab, x.v),
        STsearchByIndex(G->tab, x.w)); arco di tipo Backward
else
    if (pre[v] > pre[w])
        printf("(%s,%s):F\n",STsearchByIndex(G->tab, x.v),
            STsearchByIndex(G->tab, x.w)); arco di tipo forward
    else
        printf("(%s,%s):C\n",STsearchByIndex(G->tab, x.v),
            STsearchByIndex(G->tab, x.w)); arco di tipo cross
}
post[w] = (*time)++;
}
```

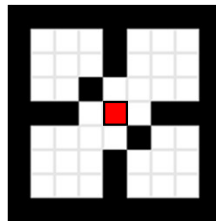
grafi orientati

## Complessità (lista adiacenze)

- Inizializzazione  $\Theta(|V|)$
- visita ricorsiva da u  $\Theta(|E|)$
- $T(n) = \Theta(|V| + |E|)$ .
- con la matrice delle adiacenze:  $T(n) = \Theta(|V|^2)$ .

## Applicazione: flood fill

- Scopo: colorare un'intera area di pixel connessi con lo stesso colore (Bucket Tool)
- DFS a partire dal pixel sorgente (seed), terminazione quando si incontra una frontiera (boundary):



<http://en.wikipedia.org>

Sedgewick, Wayne, Algorithms Part I & II, [www.coursera.org](http://www.coursera.org)

## Visita in ampiezza

A partire da un vertice  $s$ :

- determina tutti i vertici raggiungibili da  $s$ , quindi non visita necessariamente tutti i vertici a differenza della DFS
- calcola la distanza minima da  $s$  di tutti i vertici da esso raggiungibili.  caso particolare di calcolo di distanza minima per grafi NON orientati
- genera un albero della visita in ampiezza.

Ampiezza: espande tutta la frontiera tra vertici già scoperti/non ancora scoperti.

## Principi base

Scoperta di un vertice: prima volta che si incontra nella visita raggiungendolo percorrendo un arco.

Il vettore  $\text{pre}[v]$  registra il tempo di scoperta di  $v$ .

Dato un vertice  $v$ , il vettore  $\text{st}[v]$  registra il padre di  $v$  nell'albero della visita in ampiezza.

Non appena il vertice viene scoperto, si registra il tempo di scoperta e il padre nell'albero, concludendo così l'elaborazione del vertice stesso.

## Strutture dati

- grafo non pesato come matrice delle adiacenze
- coda  $Q$  di archi  $e = (v, w)$
- vettore  $st$  dei padri nell'albero di visita in ampiezza
- vettore  $pre$  dei tempi di scoperta dei vertici
- contatore  $time$  del tempo

$time$ ,  $*pre$  e  $*st$  sono locali alla funzione `GRAPHbfs`  
e passati by reference alla funzione `bfs`.

wrapper

come espando CONTEMPORANEAMENTE la frontiera?  
emulo comportamento parallelo sul modello di tipo seriale

ho bisogno di una CODA di archi

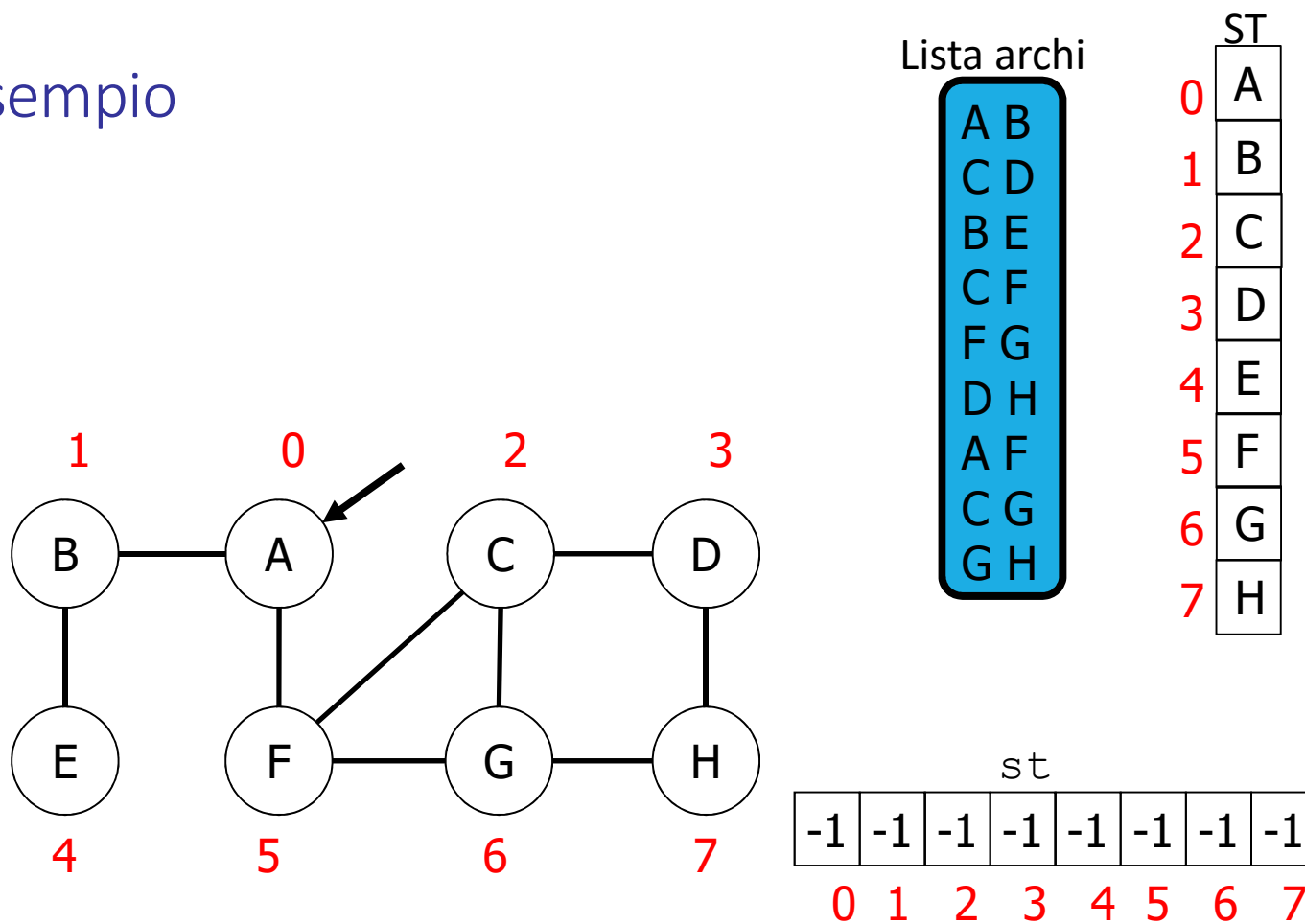
## Algoritmo

- metti l'arco fittizio di partenza  $e = (id, id)$  nella coda
- ripeti fintanto che la coda non si svuota
  - estrai dalla coda un arco  $e = (v, w)$  **che si trova in testa alla coda**
  - se  $e.w$  è un vertice non ancora scoperto ( $pre[e.w] == -1$ )
    - **lo stiamo scoprendo (è bianco)** indica che  $e.v$  è padre di  $e.w$  nella BFS ( $st[e.w] = e.v$ )
    - marca  $e.w$  come scoperto al tempo  $time$  ( $pre[e.w] = time++$ ) e incrementa il tempo
    - trova i vertici non ancora scoperti  $x$  adiacenti a  $e.w$  e metti in coda tutti gli archi che connettono  $e.w$  ad essi.

`bfs`: funzione che visita in ampiezza a partire da un vertice di partenza `id`. **in particolare tutti quelli non ancora scoperti li metto nella coda**

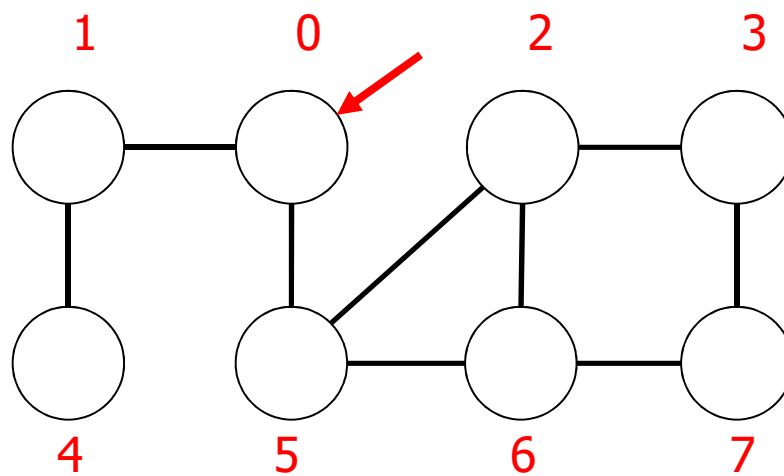


## Esempio



Q (0,0)

st

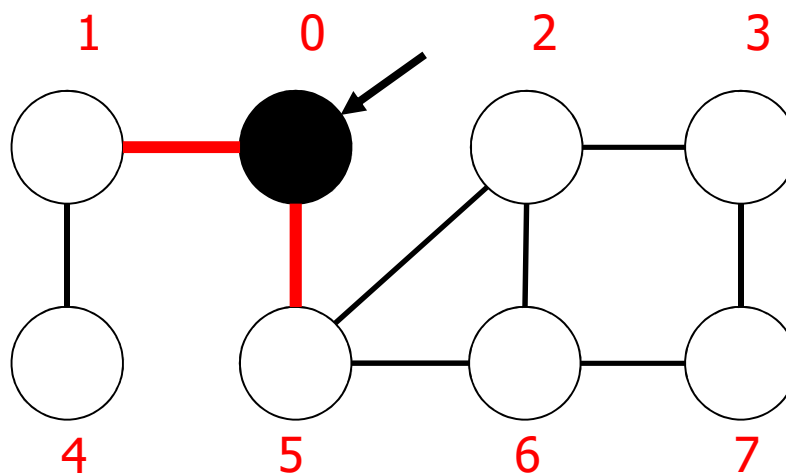


st

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |

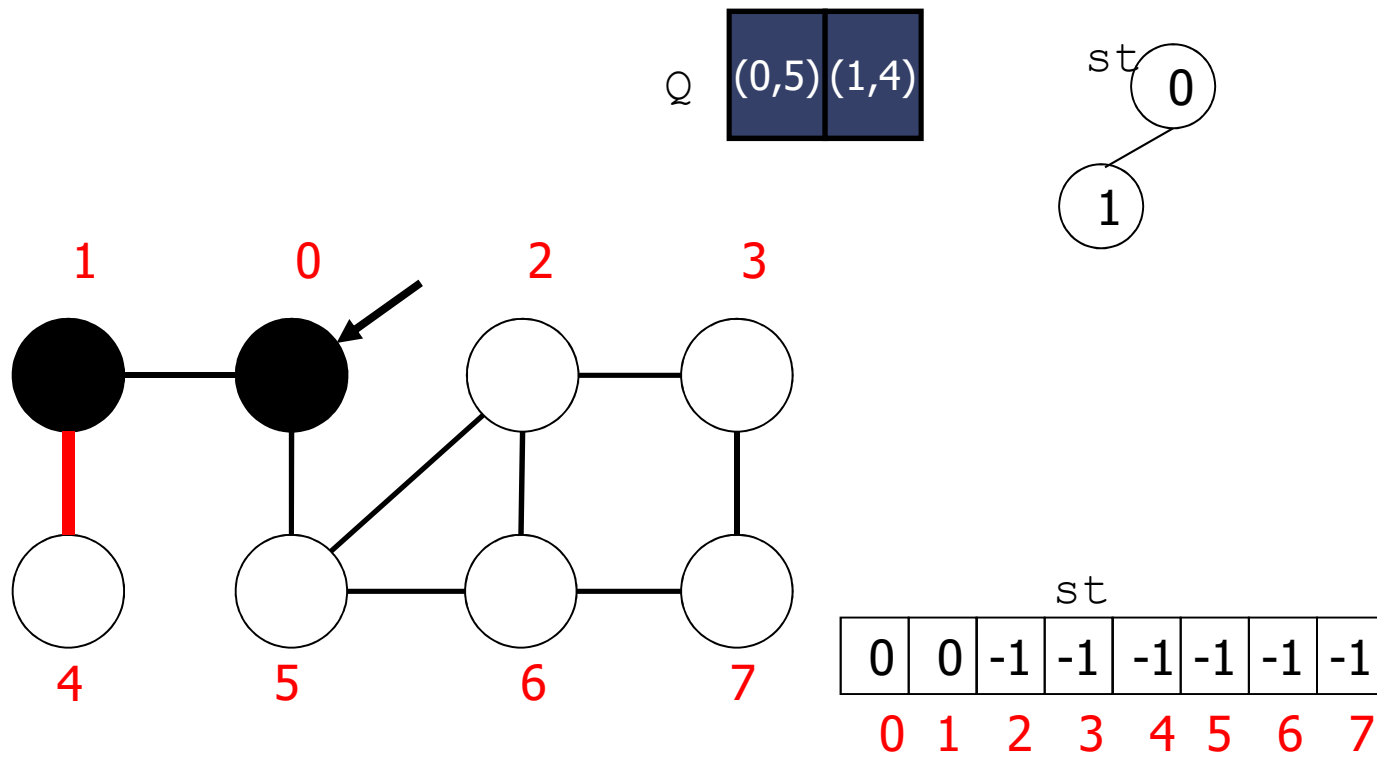
Q (0,1) (0,5)

st 0

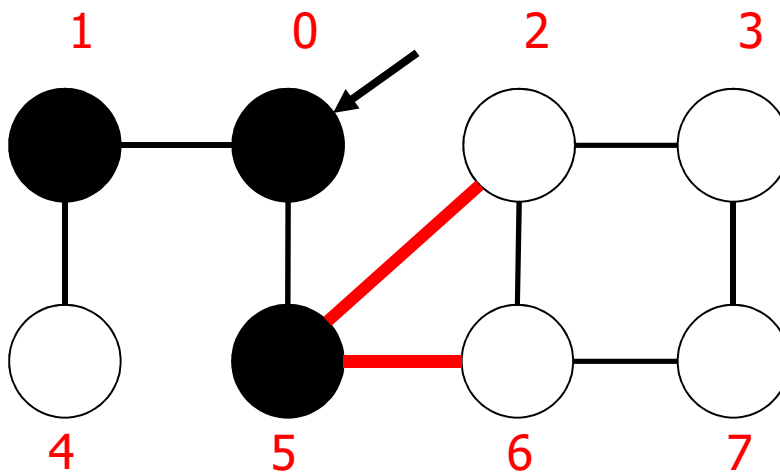


st

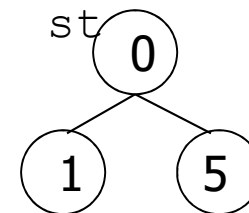
|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 0 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |



procede per fasce  
emula comportamento parallelo  
nonostante il processo sia seriale



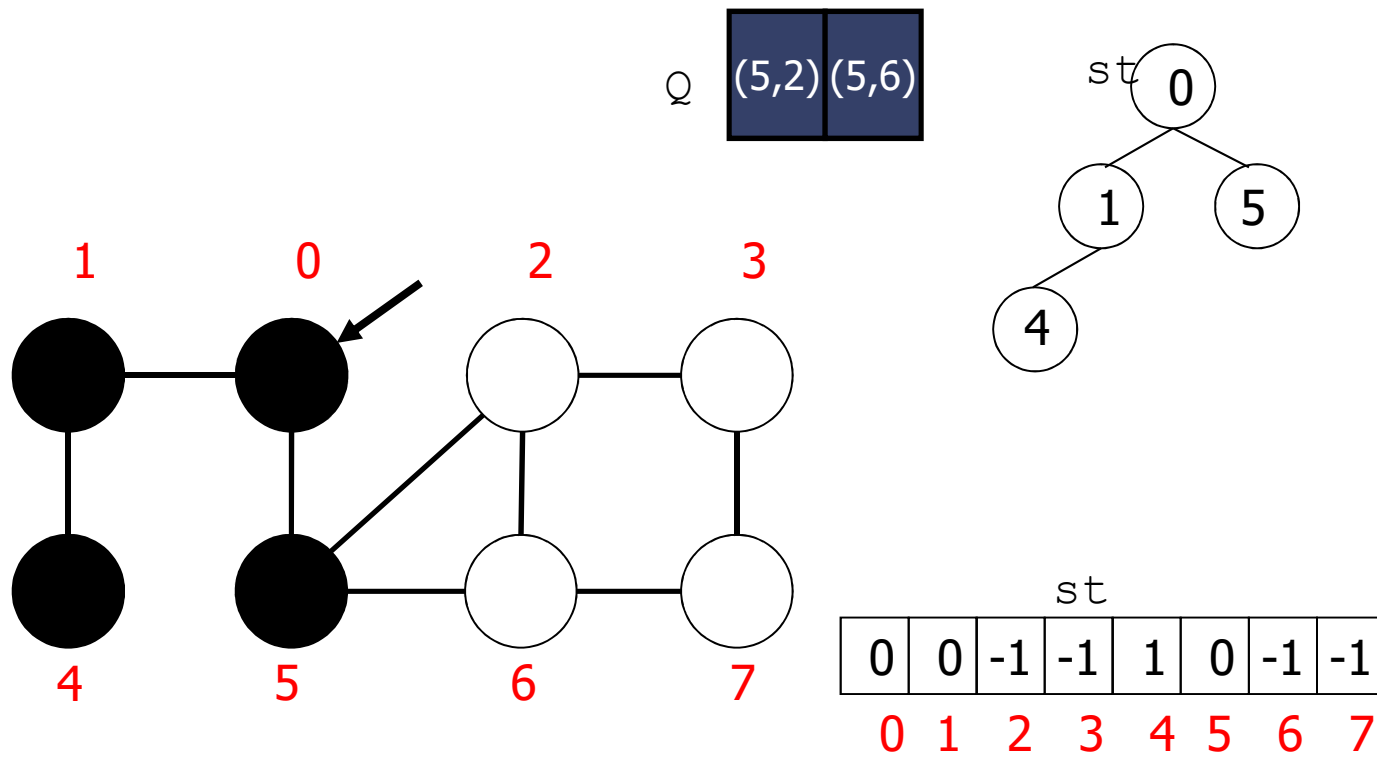
Q (1,4) (5,2) (5,6)

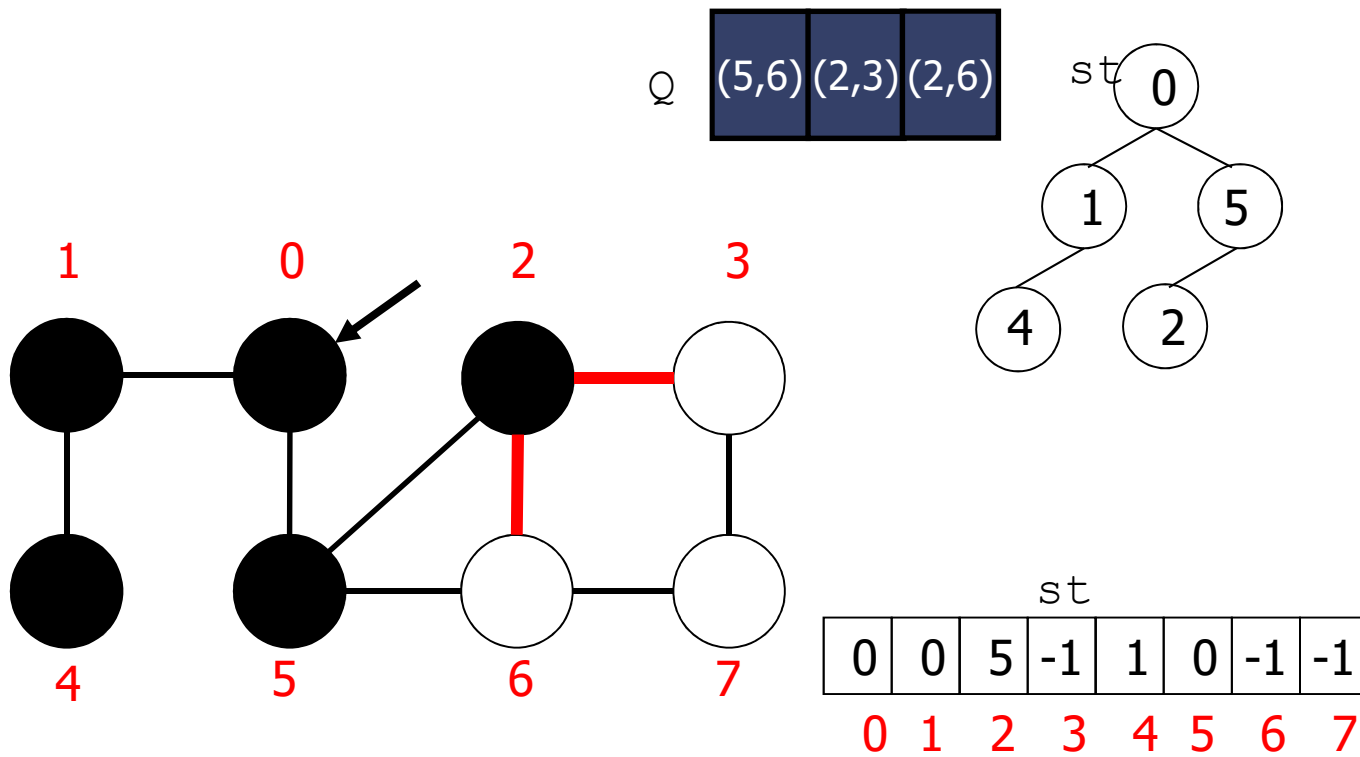


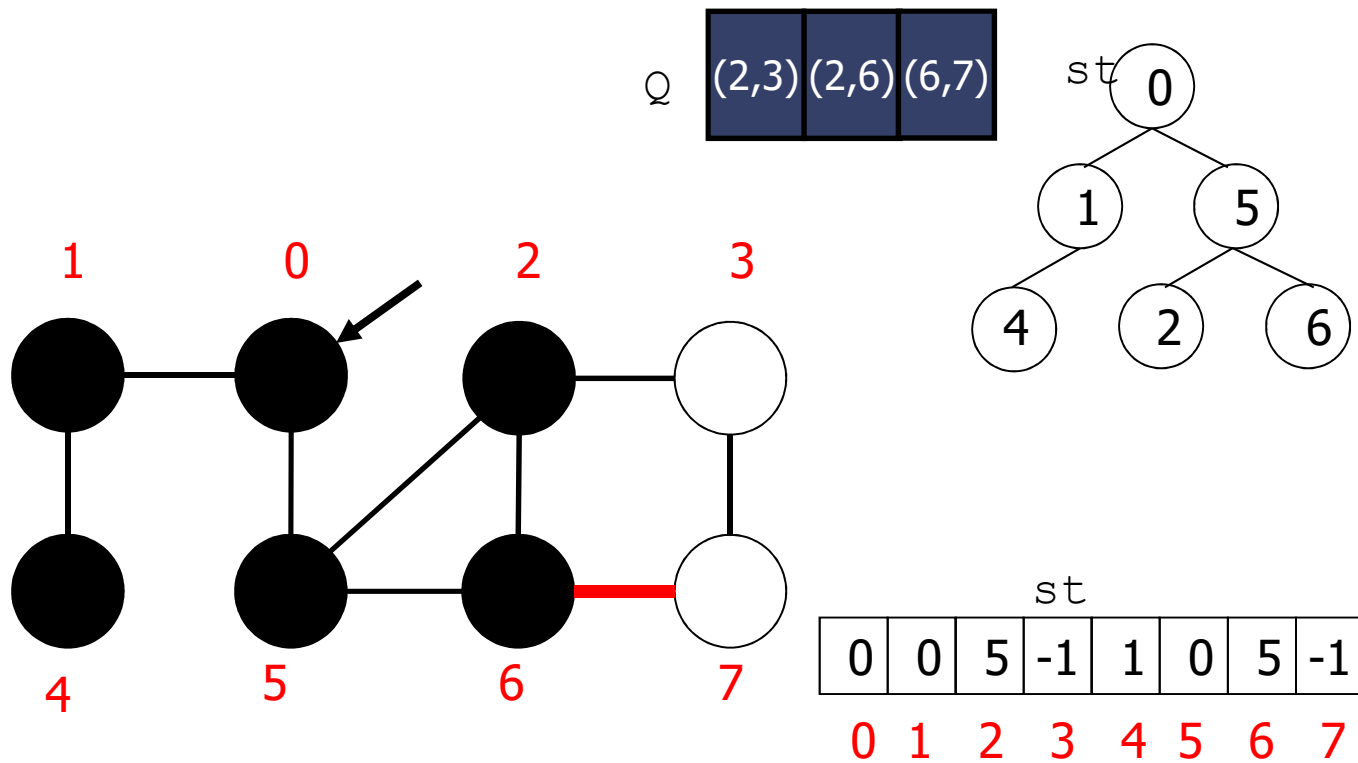
tratto prima tutti i nodi raggiungibili dal  
nodo 0 (parallelamente) e poi scendo  
di livello

tutti i nodi più in profondità di 1 e 5 vengon  
trattati DOPO aver esaurito 1 e 5

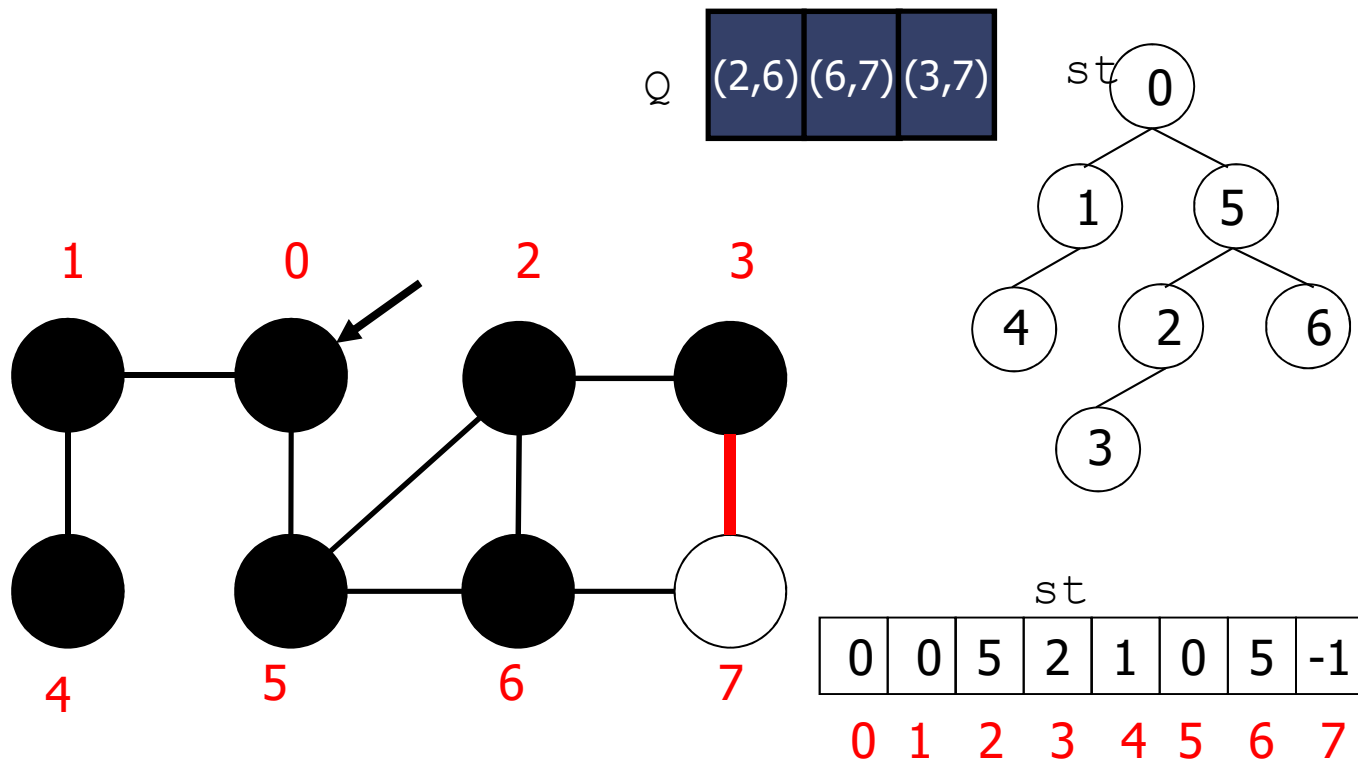
| st |   |    |    |    |   |    |    |
|----|---|----|----|----|---|----|----|
| 0  | 0 | -1 | -1 | -1 | 0 | -1 | -1 |
| 0  | 1 | 2  | 3  | 4  | 5 | 6  | 7  |

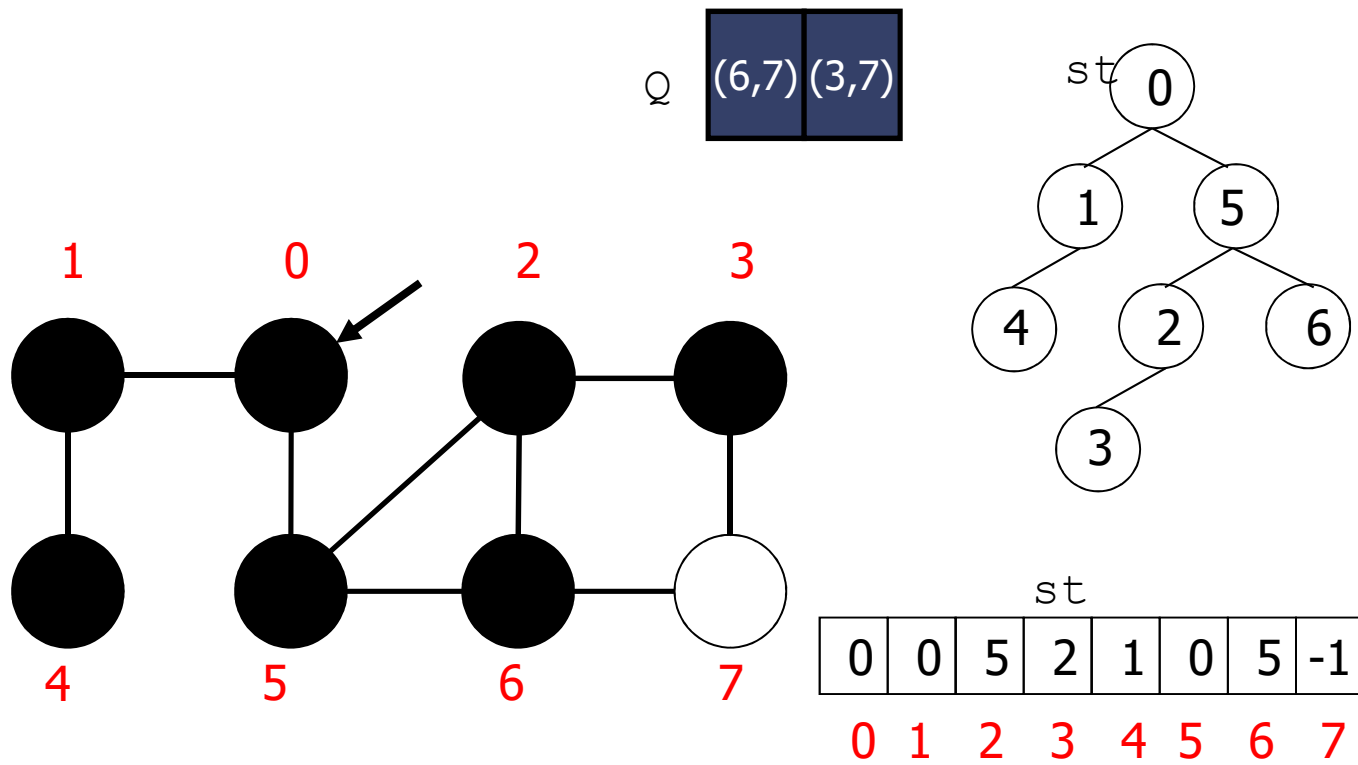


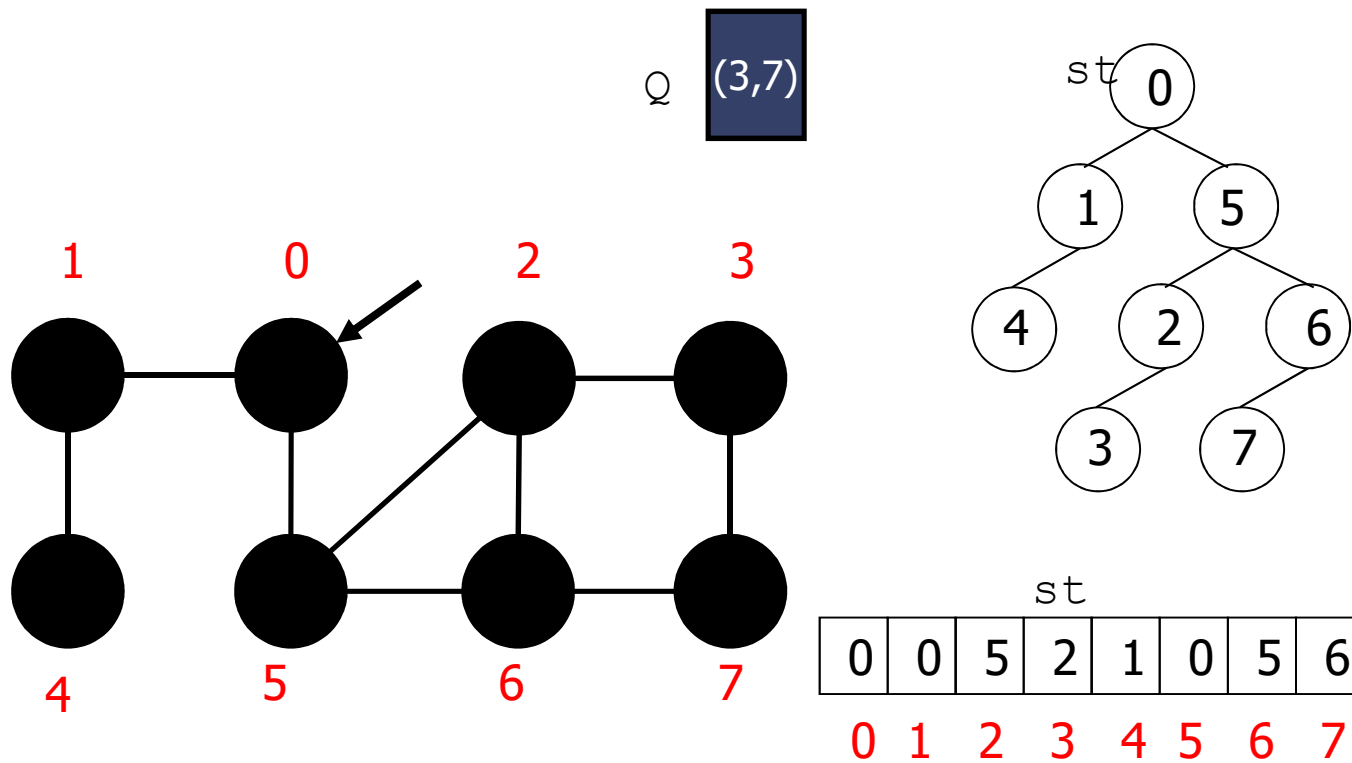


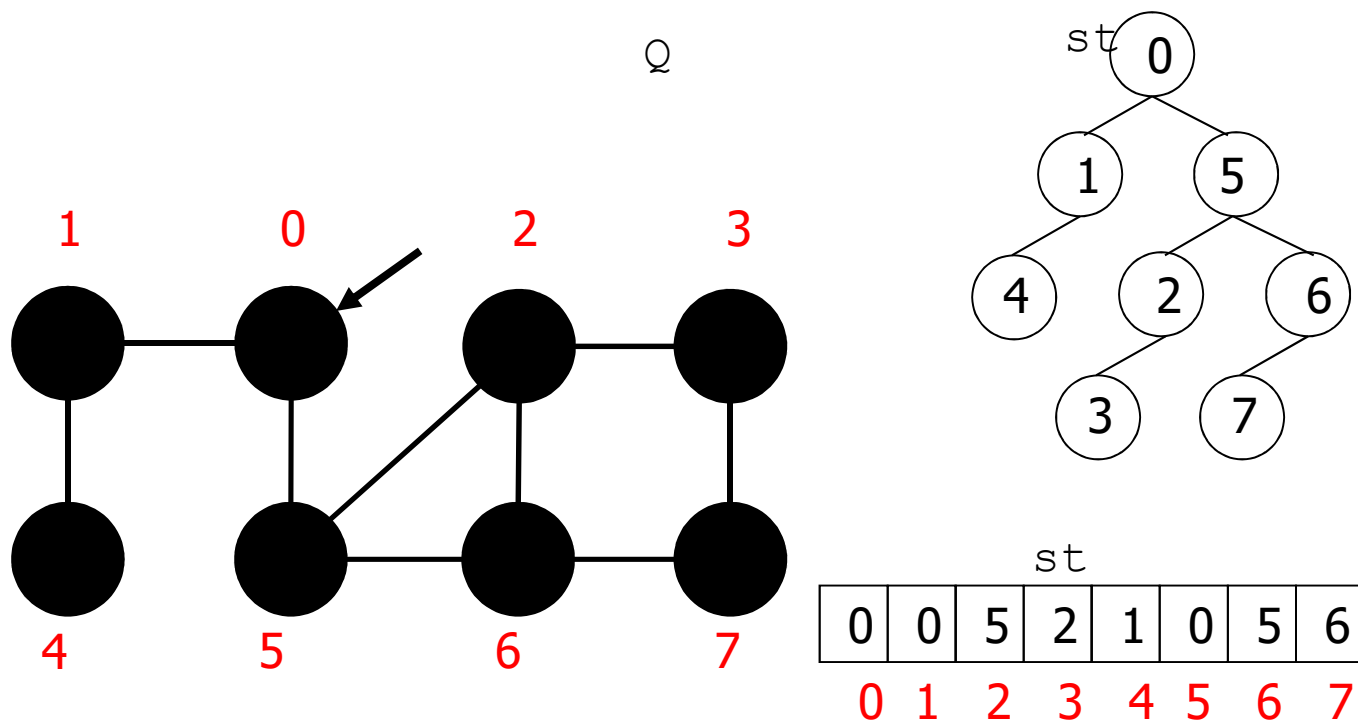












```

void GRAPHbfs(Graph G, int id) {
    int v, time=0, *pre, *st, *dist;
    /* allocazione di pre, st e dist */
    for (v=0; v < G->V; v++) {
        pre[v] = -1; st[v] = -1; dist[v] = INT_MAX;
    }
    bfs(G, EDGEcreate(id,id), &time, pre, st, dist); crea arco fittizio
    printf("\n Resulting BFS tree \n");
    for (v=0; v < G->V; v++)
        if (st[v] != -1)
            printf("%s's parent is:%s\n",STsearchByIndex(G->tab, v),
                STsearchByIndex(G->tab, st[v]));
    printf("\n Levelizing \n");
    for (v=0; v < G->V; v++)
        if (st[v] != -1)
            printf("%s: %d \n",STsearchByIndex(G->tab,v),dist[v]);
}

```

```
void bfs(Graph G, Edge e, int *time, int *pre, int *st,
        int *dist) {
```

```
    int x;
```

```
    Q q = Qinit();
```

```
    Qput(q, e);
```

```
    dist[e.v] = -1; // distanza di e.v dal nodo che stiamo trattando dato che e.v è un nodo
                  // fittizio lo inizializzo a -1
```

```
    while (!Qempty(q)) { // estraggo Qget=e (considero e.w)
```

```
        if (pre[(e = Qget(q)).w] == -1) { // è un vertice bianco?
```

```
            pre[e.w] = (*time)++; // se sì: salvo il tempo di scoperta di e.w e POI
                                // incremento il contatore
```

```
            st[e.w] = e.v;
```

```
            dist[e.w] = dist[e.v] + 1; // scandisco tutti i vertici possibili
```

```
            for (x = 0; x < G->V; x++)
```

```
                if (G->adj[e.w][x] == 1)
```

```
                    if (pre[x] == -1) // se il vertice è adiacente (dista 1)
```

```
                        Qput(q, EDGEcreate(e.w, x));
```

```
        }
```

```
    }
```

il padre di e.w è e.v  
la distanza di un nodo  
da sè stesso è 0 (-1+1)  
esamino i vertici adiacenti

matrice delle  
adiacenze

## Complessità

- Operazioni sulla coda
- Scansione della matrice delle adiacenze  $T(n) = \Theta(|V|^2)$ .
- Con la lista delle adiacenze:  $T(n) = O(|V| + |E|)$ .

Proprietà ricerca in ampiezza risolve il problema dei

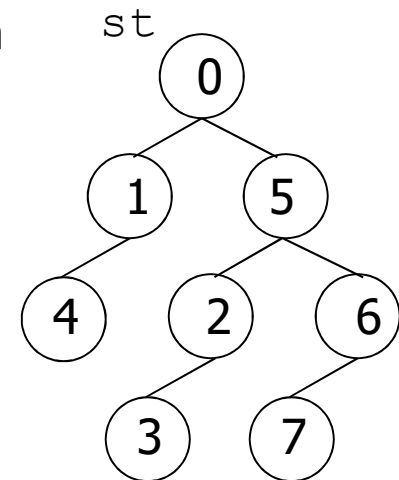
Cammini minimi: la visita in ampiezza determina la minima distanza tra s e ogni vertice raggiungibile da esso. per grafi NON pesati

come grafo pesato con tutti i pesi di pari valore

Cammino minimo da 0 a 3:

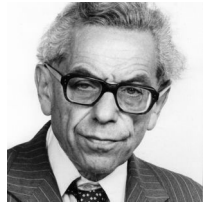
0, 5, 2, 3

lunghezza = 3

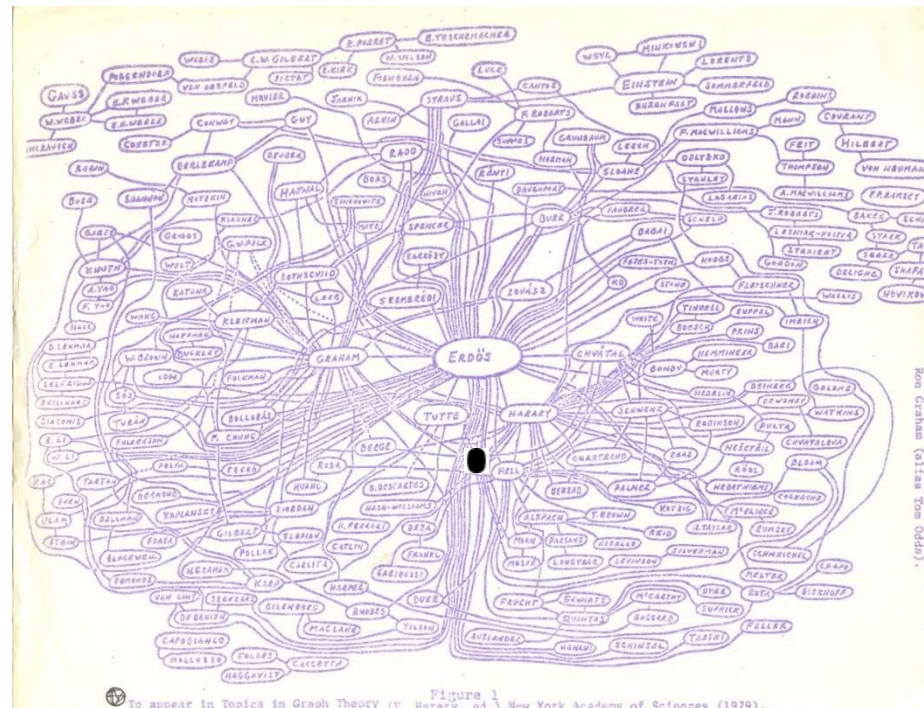




## Applicazione: I numeri di Erdős



- Paul Erdős (1913-1996): matematico ungherese «itinerante»: pubblicazioni con moltissimi coautori
- Grafo non orientato:
  - vertici: matematici
  - arco: unisce 2 matematici che hanno una pubblicazione in comune
- numero di Erdős: distanza minima di ogni matematico da Erdős
- BFS



<http://www.oakland.edu/upload/images/Erdos%20Number%20Project/cgraph.jpg>  
Sedgewick, Wayne, Algorithms Part I & II, [www.coursera.org](http://www.coursera.org)

## Riferimenti

- Visita in profondità:
  - Sedgewick Part 5 18.2, 18.3, 18.4
  - Cormen 23.3
- Visita in ampiezza:
  - Sedgewick Part 5 18.7
  - Cormen 23.2
- Numero di Erdős:
  - Bertossi 9.5.2

## Esercizi di teoria

- 10. Visite dei grafi e applicazioni
  - 10.2 Visita in ampiezza
  - 10.3 Visita in profondità

